

ΤΕΛΙΚΗ ΕΡΓΑΣΙΑ

ARCHITECTING A CLOUD-NATIVE APP USING GITOPS AND
SERVERLESS

ΔΡΟΣΟΣ ΚΑΤΣΙΜΠΡΑΣ

Μάθημα: Σχεδιασμός Cloud Native Εφαρμογών
ΠΜΣ «Προηγμένες Τεχνολογίες Πληροφορικής και Εφαρμογές»

Ακαδημαϊκά μέλη μαθήματος:
Καθηγητής κ. Κουσιουρής Γεώργιος
Καθηγητής κ. Τσαδήμας Ανάργυρος



ΧΑΡΟΚΟΠΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ
ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΜΑΤΙΚΗΣ

Τμήμα Πληροφορικής και Τηλεματικής
Χαροκόπειο Πανεπιστήμιο Αθηνών
Ιούνιος, 2026

ΔΡΟΣΟΣ ΚΑΤΣΙΜΠΡΑΣ

Περίληψη

Η μετάβαση από παραδοσιακές, τοπικά φιλοξενούμενες υποδομές σε δυναμικά περιβάλλοντα νέφους αποτελεί επιτακτική ανάγκη για την εξασφάλιση της κλιμακωσιμότητας και της ανθεκτικότητας των σύγχρονων εφαρμογών. Η παρούσα εργασία παρουσιάζει την αρχιτεκτονική εξέλιξη ενός συστήματος Διαχείρισης Έξυπνης Αποθήκης (Smart Warehouse ERP) – το οποίο αρχικά βασιζόταν σε μια στατική, χειροκίνητα αναπτυσσόμενη υποδομή – σε μια πλήρως αυτοματοποιημένη, Cloud-Native πλατφόρμα. Ο κύριος στόχος είναι η επίλυση προβλημάτων περιορισμού πόρων, απώλειας δεδομένων και μη αυτοματοποιημένων κύκλων ανάπτυξης (deployments), μέσω της υιοθέτησης βέλτιστων πρακτικών του κλάδου της Μηχανικής Λογισμικού (DevOps).

Για την επίτευξη αυτού του μετασχηματισμού, υιοθετήθηκε η μεθοδολογία Infrastructure as Code (IaC) μέσω των εργαλείων OpenTofu και Ansible, εξασφαλίζοντας την αυτόματη και την εγκαθίδρυση της υποδομής σε περιβάλλον Microsoft Azure. Ο κύκλος ζωής των εφαρμογών (microservices) ενορχηστρώνεται βάσει της φιλοσοφίας GitOps, συνδυάζοντας τον Jenkins για το Continuous Integration - (CI) και το ArgoCD για το Continuous Deployment - (CD) εντός ενός MicroK8s cluster. Η επιχειρηματική λογική ενισχύθηκε με μοτίβα σχεδιασμού ανθεκτικότητας (Design Patterns), όπως Idempodency και Retry, ενώ ενσωματώθηκε serverless υπολογιστική λογική για την ασύγχρονη, event-driven επεξεργασία δεδομένων, δημιουργώντας thumbnails των εικόνων. Παράλληλα, εξασφαλίστηκε η μονιμότητα των δεδομένων (Data Persistence) μέσω Managed Disks και επιτεύχθηκε κεντρική διαχείριση μυστικών μέσω του Hashicorp Vault.

Τέλος, το σύστημα θωρακίστηκε με μηχανισμούς οπτικοποίησης και παρακολούθησης (Observability) με χρήση Prometheus και Grafana. Βάσει πειραματικών δοκιμών φόρτου (Load Testing), εξήχθησαν κανόνες κλιμάκωσης και διαμορφώθηκαν ρεαλιστικές Συμφωνίες Επιπέδου Υπηρεσιών (SLAs), ταξινομημένες σε κατηγορίες Gold, Silver και Bronze. Συνολικά, η εργασία αποδεικνύει εμπράκτως πώς η σύγκλιση των μεθοδολογιών GitOps, IaC και Cloud-Native αρχιτεκτονικών μεταμορφώνει ένα ακαδημαϊκό πρωτότυπο σε μια παραγωγικά έτοιμη (production-grade), πλήρως ανθεκτική πλατφόρμα.

Λέξεις Κλειδιά: Cloud-Native Architecture, GitOps, Infrastructure as Code (IaC), Continuous Integration and Deployment (CI/CD), Microservices Resilience, Serverless Computing, Service Level Agreements (SLA), Kubernetes Observability.

Περιεχόμενα

Περίληψη	ii
Κατάλογος Σχημάτων	vii
Κατάλογος Πινάκων	ix
Λίστα Συντομογραφιών	xi
1 Κεφάλαιο 1: Εισαγωγή και σκοπιμότητα	1
1.1 Αντικείμενο της εργασίας	1
1.2 Σκοπιμότητα και σενάριο χρήσης (<i>Business Scenario</i>)	1
1.3 Στόχοι του <i>project</i>	2
1.4 Μεθοδολογία υλοποίησης	2
2 Κεφάλαιο 2: Αρχιτεκτονική σεναρίου και χρήση υπηρεσιών	3
2.1 Το σενάριο: <i>Smart Warehouse ERP</i>	3
2.2 Εξέλιξη αρχιτεκτονικής (<i>Evolution</i>)	4
2.3 Ανάλυση <i>Microservices</i>	6
2.4 Τεχνολογικά κριτήρια επιλογής	6
2.5 Μόνιμη αποθήκευση δεδομένων (<i>Data Persistence</i>)	7
3 Κεφάλαιο 3: Αυτοματοποίηση υποδομών και ασφάλεια	9
3.1 <i>Infrastructure as Code</i> με <i>OpenTofu</i>	9
3.2 Αυτοματοποιημένη εγκατάσταση με <i>Ansible</i>	11
3.3 Εκτίμηση κόστους υποδομής (<i>FinOps</i>): <i>Infracost</i>	13
3.4 Διαχείριση των <i>secrets</i> με <i>HashiCorp Vault</i>	13
4 Κεφάλαιο 4: <i>GitOps</i> και <i>CI/CD Pipelines</i>	16
4.1 Δομή του <i>Git Monorepo</i>	16
4.2 <i>Continuous Integration</i> - (<i>CI</i>) με <i>Jenkins</i>	16
4.2.1 Αυτόματη ενεργοποίηση μέσω <i>webhook</i> και επιλεκτική εκτέλεση	17
4.2.2 Το <i>Jenkinsfile</i> και η διαδικασία <i>Build & Test</i>	17
4.2.3 <i>Push</i> των <i>container images</i> στο <i>Docker Hub</i>	19
4.3 <i>Continuous Delivery & Deployment</i> - (<i>CD</i>) με <i>ArgoCD</i>	19
5 Κεφάλαιο 5: Μοτίβα σχεδιασμού και ανθεκτικότητα εφαρμογής	22
5.1 <i>Claim Check Pattern</i>	22
5.2 <i>Retry Pattern</i>	23
5.3 <i>Idempotency Pattern</i>	23

5.4	<i>Stateless Pattern</i>	24
6	Κεφάλαιο 6: Serverless Υπολογισμοί και λογική αυτοματοποίησης	26
6.1	Εγκατάσταση <i>serverless</i> πλατφόρμας: <i>Knative</i>	26
6.2	Σχεδιασμός <i>triggering</i> μηχανισμού	26
6.3	Εκτέλεση επιχειρηματικής λογικής: (<i>thumbnails</i>)	27
6.3.1	Διάγνωση και επίλυση <i>infinite loop</i> σε <i>event-driven</i> λογική	27
7	Κεφάλαιο 7: Παρακολούθηση, κλιμάκωση και SLAs	30
7.1	Οπτικοποίηση και μετρικές (<i>Observability</i>): <i>Prometheus & Grafana</i>	30
7.2	Αυτόματη κλιμάκωση (<i>HPA — Horizontal Pod Autoscaler</i>)	32
7.3	Πειραματικές μετρήσεις και <i>load testing</i>	33
7.4	Καθορισμός επιπέδων ποιότητας υπηρεσίας (<i>SLA Levels</i>)	36
7.4.1	Ανάλυση κατανομής (<i>99% CDF</i>)	37
7.4.2	Ορισμός κλάσεων <i>SLA</i> (<i>Gold, Silver, Bronze</i>) βάσει χρόνου απόκρισης	37
	Συμπεράσματα και Μελλοντικές επεκτάσεις	37
	Παράρτημα Α: Τεχνική Αναφορά	39
	Παράρτημα Β: Αποθετήριο Κώδικα (GitHub)	43
	Βιβλιογραφικές Αναφορές	45

Κατάλογος Σχημάτων

2.1	Εξέλιξη: από το <i>MicroK8s</i> τοπικής εγκατάστασης (<i>current state</i>) στο πλήρως αυτοματοποιημένο <i>cloud-native</i> περιβάλλον στο <i>Azure</i> (<i>target state</i>)	5
3.1	Ροή IaC: OpenTofu provisioning	9
3.2	Αποτέλεσμα του OpenTofu provisioning	11
3.3	Ansible: Run the playbook για κάθε role	12
3.4	Ανάλυση κόστους υποδομής μέσω <i>Infracost</i> : εκτιμώμενο μηνιαίο κόστος \$133.03	13
3.5	HashiCorp Vault: αποθηκευμένα μυστικά εφαρμογής στο <i>path secret/fabric</i>	15
4.1	GitOps pipeline: από το git push στο cluster μέσω Jenkins και ArgoCD	16
4.2	Επιτυχής εκτέλεση του <i>Jenkins pipeline</i>	18
4.3	<i>ArgoCD</i> : εφαρμογή σε κατάσταση <i>Healthy</i> και <i>Synced</i> μετά από αυτόματο συγχρονισμό	20
5.1	Δοκιμή του <i>Idempotency Pattern</i> μέσω <i>curl</i> : το δεύτερο αίτημα με το ίδιο <i>idempotency key</i> απορρίπτεται με 409 <i>Conflict</i>	24
6.1	Αρχεία καταγραφής (<i>logs</i>) της συνάρτησης <i>Knative</i> μετά τη διόρθωση: το αρχείο επεξεργάζεται μία φορά (« <i>Thumbnail created</i> »), και το επόμενο γεγονός <i>PutTagging</i> αναγνωρίζεται σωστά ως ήδη επεξεργασμένο (« <i>Already processed</i> »), χωρίς νέο κύκλο εκτέλεσης	28
7.1	<i>Grafana</i> : προσαρμοσμένο <i>dashboard</i> παρακολούθησης <i>CPU</i> και μνήμης	30
7.2	<i>Grafana</i> : προβολή επεξεργασίας (<i>edit panel</i>) του ερωτήματος <i>PromQL</i> για τη χρήση <i>CPU</i> του <i>backend</i>	31
7.3	<i>Grafana</i> : <i>dashboard "Node Exporter Full"</i> — συνολική χρήση πόρων της εικονικής μηχανής <i>fabric-vm</i>	32
7.4	Αποτελέσματα δοκιμής φόρτου <i>k6</i> για το <i>endpoint GET /api/products</i> : 6.823 αιτήματα, p(95)=14.2ms, μηδενικό ποσοστό αποτυχίας	34
7.5	Αποτελέσματα δοκιμής φόρτου <i>k6</i> για το <i>endpoint POST /api/products</i> : 1.018 αιτήματα, p(95)=80.13ms, μηδενικό ποσοστό αποτυχίας	35
7.6	Αποτελέσματα δοκιμής φόρτου <i>k6</i> για το <i>endpoint POST /api/products</i> : 1.018 αιτήματα, p(95)=80.13ms, μηδενικό ποσοστό αποτυχίας	36

Κατάλογος Πινάκων

1	Application-level services and technology choices	7
2	Συνοπτικός πίνακας εντολών λειτουργίας συστήματος	40
3	Σημεία πρόσβασης στις διαχειριστικές διεπαφές του συστήματος	40
4	Κύριες τεχνολογίες υποδομής και αντίστοιχες εκδόσεις	41
5	Σύνολο <i>Ansible roles</i> και η αντίστοιχη ευθύνη τους	41
6	Σύνδεσμοι <i>repository</i>	43

Λίστα Συντομογραφιών

API	Application Programming Interface
CD	Continuous Delivery / Continuous Deployment
CDF	Cumulative Distribution Function
CI	Continuous Integration
ERP	Enterprise Resource Planning
FQDN	Fully Qualified Domain Name
HPA	Horizontal Pod Autoscaler
HTTP	HyperText Transfer Protocol
IaC	Infrastructure as Code
IoT	Internet of Things
JSON	JavaScript Object Notation
K8s	Kubernetes
NSG	Network Security Group
PV	Persistent Volume
PVC	Persistent Volume Claim
RBAC	Role-Based Access Control
S3	Simple Storage Service
SLA	Service Level Agreement
SLI	Service Level Indicator
SQL	Structured Query Language
SSH	Secure Shell
SSO	Single Sign-On
VM	Virtual Machine

1 Κεφάλαιο 1: Εισαγωγή και σκοπιμότητα

Σε αυτό το αρχικό κεφάλαιο παρουσιάζουμε το πλαίσιο μέσα στο οποίο αναπτύχθηκε η εργασία, την αφετηρία της (μια ήδη υπάρχουσα containerized εφαρμογή), και τη σκοπιμότητα της μετάβασής της σε *cloud-native* περιβάλλον. Στη συνέχεια καθορίζονται οι συγκεκριμένοι στόχοι της εργασίας και η μεθοδολογία με την οποία προσεγγίστηκε η υλοποίησή τους.

1.1 Αντικείμενο της εργασίας

Αρχικά, η παρούσα εργασία αποτελεί επέκταση μιας εφαρμογής που αναπτύχθηκε στο πλαίσιο του μαθήματος «*Cloud Platforms*», όπου η εφαρμογή είχε ήδη μεταφερθεί σε containerized μορφή (Docker) και λειτουργούσε σε MicroK8s, εγκατεστημένο τοπικά στο μηχάνημα ανάπτυξης. Αν και το περιβάλλον αυτό κάλυπτε τις βασικές ανάγκες containerization και ορχήστρωσης, παρέμεναν σημαντικοί περιορισμοί: η υποδομή ήταν δεμένη με το συγκεκριμένο τοπικό μηχάνημα, χωρίς δυνατότητα ανεξάρτητης κλιμάκωσης ή απομακρυσμένης πρόσβασης, η ρύθμιση του περιβάλλοντος γινόταν χειροκίνητα και δεν ήταν αναπαραγώγιμη, οι αλλαγές στον κώδικα απαιτούσαν χειροκίνητη παρέμβαση για να φτάσουν σε λειτουργία, και δεν υπήρχε κανένας μηχανισμός διαχείρισης μυστικών (secrets), παρακολούθησης απόδοσης, ή ανθεκτικότητας σε σφάλματα.

Συνεπώς το αντικείμενο της παρούσας εργασίας είναι η μετάβαση αυτής της ήδη containerized εφαρμογής σε πλήρως αυτοματοποιημένη, cloud-native υποδομή: παροχή της απαιτούμενης υποδομής σε δημόσιο cloud πάροχο (Azure) μέσω **Infrastructure as Code**, αυτοματοποιημένη ρύθμιση του περιβάλλοντος, ενσωμάτωση πρακτικών **GitOps**, σχεδιαστικών προτύπων ανθεκτικότητας, **serverless** υπολογισμού, και τεκμηριωμένων επιπέδων ποιότητας υπηρεσίας.

1.2 Σκοπιμότητα και σενάριο χρήσης (*Business Scenario*)

Το λειτουργικό σενάριο της εφαρμογής παραμένει το ίδιο με αυτό που είχε υλοποιηθεί: πρόκειται για ένα σύστημα διαχείρισης αποθήκης (ERP-style), όπου όταν ένας χρήστης προσθέτει νέο προϊόν μέσω του *frontend*, η εικόνα του προϊόντος αποθηκεύεται στο *MinIO* (object storage) και ενεργοποιείται μια *serverless function* που παράγει ένα *thumbnail* της εικόνας, έπειτα παράγεται μήνυμα ειδοποίησης στο *RabbitMQ*, και η πληροφορία προχωράει στο *Node-RED*, όπου προσομοιώνονται περιβαλλοντικές τιμές (θερμοκρασία, υγρασία) που αντιστοιχούν στις συνθήκες αποθήκευσης του προϊόντος. Οι τιμές αυτές απεικονίζονται σε dashboard, ώστε ο διαχειριστής της αποθήκης να έχει εικόνα σε πραγματικό χρόνο για τις περιβαλλοντικές συνθήκες κάθε αποθηκευτικού χώρου.

Η σκοπιμότητα λοιπόν, έγκειται στη μετάβαση αυτού του ήδη λειτουργικού σεναρίου από μια αυτόνομη, τοπική εγκατάσταση σε μια πλήρως cloud-native αρχιτεκτονική. Το σύστημα τρέχει πλέον σε εικονική μηχανή Azure που δημιουργείται μέσω OpenTofu, με τη ρύθμιση του περιβάλλοντος (Kubernetes, διαχείριση μυστικών, παρακολούθηση, εργαλεία CI/CD) να αυτοματοποιείται πλήρως μέσω **Ansible**. Η ανάπτυξη της εφαρμογής ακολουθεί πρακτικές GitOps (Jenkins για continuous integration, ArgoCD για continuous deployment), ενώ ο κώδικας της εφαρμογής ενσωματώνει σχεδιαστικά πρότυπα cloud-native εφαρμογών (design patterns) και έχει αξιολογηθεί μέσω δοκιμών φόρτου για τον καθορισμό επιπέδων SLA.

1.3 Στόχοι του *project*

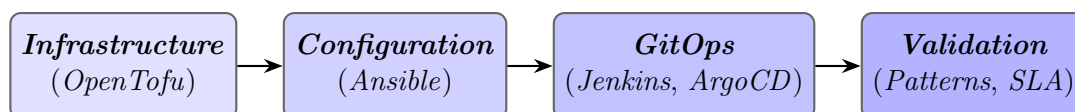
Σε αυτή την ενότητα και δεδομένων των περιορισμών που περιγράφηκαν παραπάνω, η εργασία δεν επιδιώκει μόνο τη μεταφορά της εφαρμογής σε cloud περιβάλλον, αλλά την υιοθέτηση ενός συνόλου πρακτικών που να καθιστούν το σύστημα ανθεκτικό, αυτοματοποιημένο, και μετρήσιμο. Η επιλογή των συγκεκριμένων τεχνολογιών και πρακτικών δεν είναι ανεξάρτητη μεταξύ τους, αλλά συνθέτει μια ολοκληρωμένη cloud-native προσέγγιση, όπου κάθε στοιχείο καλύπτει μια διαφορετική πτυχή της λειτουργίας του συστήματος. Συγκεκριμένα, οι στόχοι της εργασίας είναι:

- Ο σχεδιασμός αρχιτεκτονικής που να αξιοποιεί πολλαπλές cloud τεχνολογίες, καθεμία επιλεγμένη για συγκεκριμένο λειτουργικό ρόλο.
- την ενσωμάτωση σχεδιαστικών προτύπων ανθεκτικότητας (*design patterns*) στον κώδικα της εφαρμογής.
- παράλληλα την υλοποίηση τμήματος υπολογισμού σε *serverless* μορφή.
- την πλήρη αυτοματοποίηση της υποδομής μέσω *Infrastructure as Code* και *configuration management* εργαλείων.
- εφαρμογή πρακτικών *GitOps* για τον στόχο του *CI/CD*.
- και την τεκμηρίωση επιπέδων ποιότητας υπηρεσίας (*SLA*) με βάση πειραματικές μετρήσεις φόρτου.

1.4 Μεθοδολογία υλοποίησης

Τέλος κλείνοντας το πρώτο κεφάλαιο, είναι σημαντικό να αναφερθεί η σειρά των φάσεων υλοποίησης, όπου δεν ήταν τυχαία, αλλά ο συνδιασμός από τις εξαρτήσεις μεταξύ των επιμέρους τεχνολογιών: η ύπαρξη υποδομής (*Infrastructure*) αποτελεί προαπαιτούμενο για τη ρύθμισή της (*Configuration*), η ρύθμιση του περιβάλλοντος είναι προαπαιτούμενο για να λειτουργήσουν τα εργαλεία συνεχούς ολοκλήρωσης και ανάπτυξης (*GitOps*), και μόνο όταν το σύστημα είναι πλήρως λειτουργικό μπορούν να πραγματοποιηθούν αξιόπιστες μετρήσεις απόδοσης και επικύρωσης (*Validation*). Ακολουθώντας αυτή τη λογική, έχουμε μια σταδιακή προσέγγιση: αρχικά σχεδιάστηκε και υλοποιήθηκε η υποδομή στο [Azure](#) μέσω *OpenTofu*, ακολούθησε η αυτοματοποιημένη ρύθμιση του περιβάλλοντος μέσω *Ansible*, και τέλος ενσωματώθηκαν τα εργαλεία *GitOps*, τα *design patterns* στον κώδικα, και οι δοκιμές φόρτου.

Η δομή της παρούσας αναφοράς ακολουθεί αυτή τη σειρά εργασίας.



2 Κεφάλαιο 2: Αρχιτεκτονική σεναρίου και χρήση υπηρεσιών

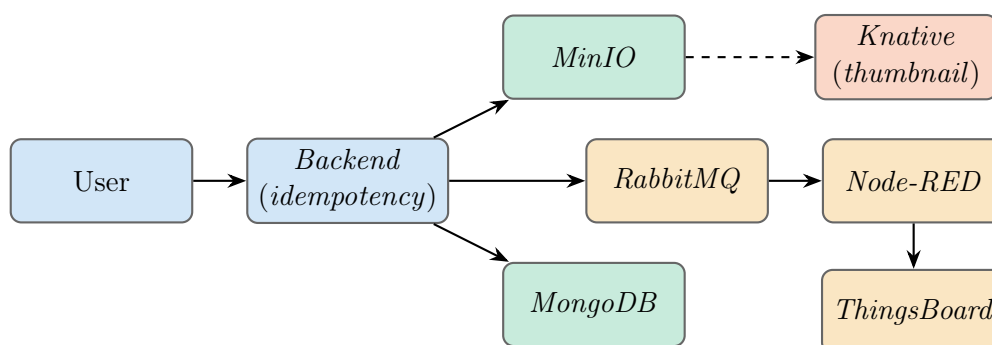
Το παρόν κεφάλαιο παρουσιάζει το σχεδιασμό της αρχιτεκτονικής σε επίπεδο συστήματος (*high-level design*): το σενάριο χρήσης και τη ροή δεδομένων που το υλοποιεί, την εξέλιξη από την προηγούμενη, τοπική αρχιτεκτονική στη σημερινή cloud-native μορφή, και την ανάλυση των επιμέρους μικροπηρεσιών που απαρτίζουν το σύστημα. Σκοπός: Το high-level design. Εδώ μπαίνουν τα διαγράμματα που συζητούσαμε.

2.1 Το σενάριο: *Smart Warehouse ERP*

Το σύστημα υλοποιεί ένα σενάριο διαχείρισης αποθήκης, όπου η κεντρική επιχειρηματική ενέργεια είναι η προσθήκη νέου προϊόντος. Η ροή δεδομένων που ακολουθείται είναι η εξής: ο χρήστης υποβάλλει τα στοιχεία του προϊόντος (όνομα, κατηγορία, τιμή, απόθεμα) μαζί με μια εικόνα, μέσω του *frontend*. Το *backend* παραλαμβάνει το αίτημα, ελέγχει αν πρόκειται για διπλή υποβολή μέσω μηχανισμού *idempotency*, και στη συνέχεια αποθηκεύει την εικόνα απευθείας στο *MinIO* (*object storage*), αντί να την περάσει ως μέρος του μηνύματος προς τις επόμενες υπηρεσίες.

Μόλις ολοκληρωθεί η αποθήκευση, το *backend* καταγράφει το νέο προϊόν στη βάση δεδομένων (*MongoDB*) και αποστέλλει ένα ελαφρύ μήνυμα ειδοποίησης — που περιέχει μόνο την αναφορά (*URL*) της εικόνας, όχι το ίδιο το αρχείο — στην ουρά μηνυμάτων *RabbitMQ*. Το μήνυμα αυτό καταναλώνεται από το *Node-RED*, όπου προσομοιώνονται περιβαλλοντικές τιμές (θερμοκρασία, υγρασία) που αντιστοιχούν στις συνθήκες αποθήκευσης του προϊόντος, και απεικονίζονται σε *dashboard* παρακολούθησης.

Παράλληλα, η αποθήκευση της εικόνας στο *MinIO* λειτουργεί ως σημείο ενεργοποίησης (*trigger*) για μια *serverless* συνάρτηση (*Knative*), η οποία παράγει αυτόματα μικρογραφία (*thumbnail*) της εικόνας, χωρίς να απαιτείται μόνιμα ενεργό *compute* για αυτή τη λειτουργία.



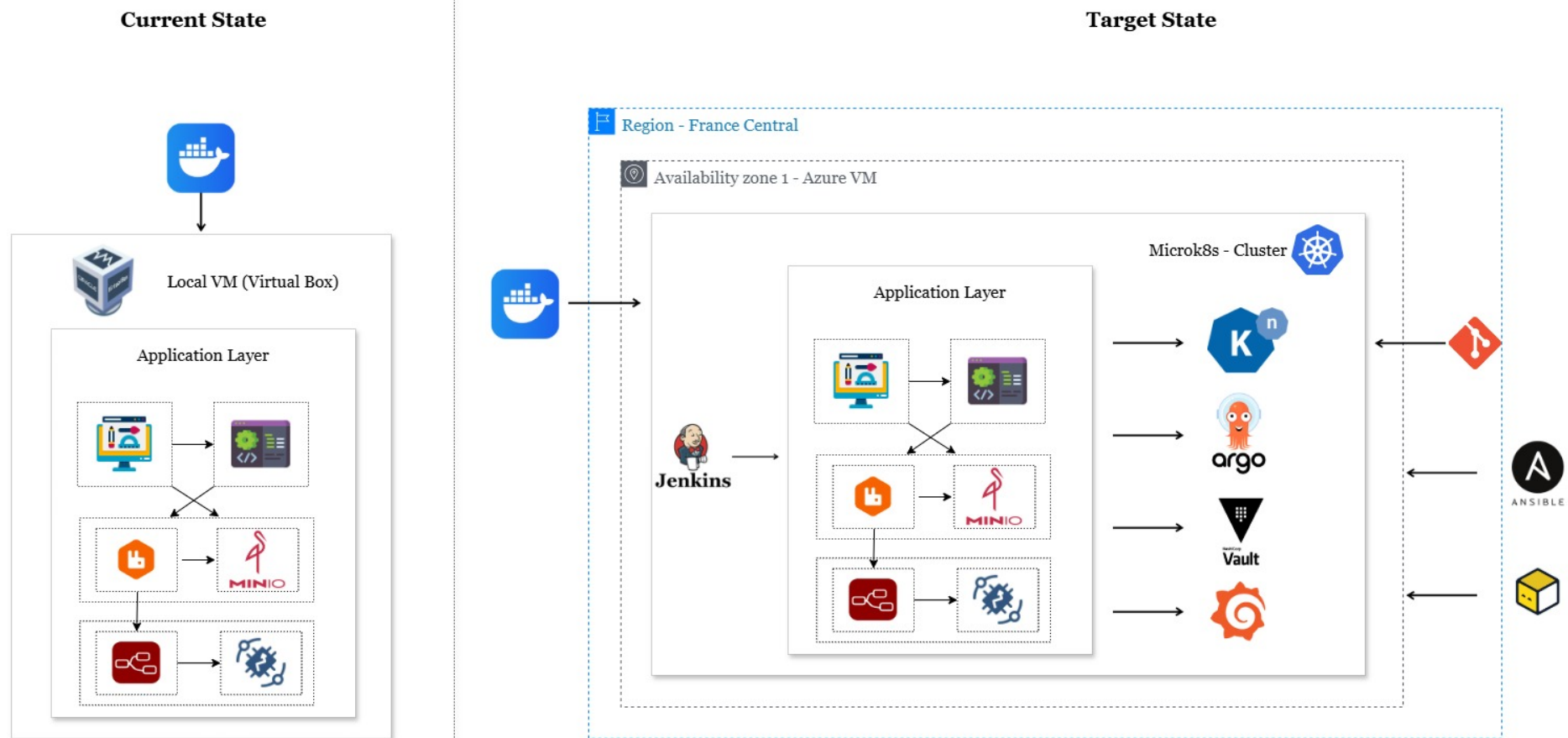
2.2 Εξέλιξη αρχιτεκτονικής (*Evolution*)

Η αρχιτεκτονική του συστήματος δεν προέκυψε από το μηδέν, αλλά αποτελεί την εξέλιξη μιας ήδη *containerized* εφαρμογής. Σε προηγούμενη εργασία, η εφαρμογή είχε ήδη μεταφερθεί σε *containerized* μορφή μέσω *Docker* και αναπτυχθεί σε *MicroK8s*, εγκατεστημένο τοπικά στο μηχάνημα ανάπτυξης.

Το παρόν στάδιο εξέλιξης — αντικείμενο της παρούσας εργασίας — δεν αγγίζει την επιχειρηματική λογική της εφαρμογής, αλλά αναδιαμορφώνει πλήρως τον τρόπο με τον οποίο αυτή παρέχεται, λειτουργεί, και συντηρείται. Συγκεκριμένα, η μετάβαση αυτή επιφέρει τις ακόλουθες αλλαγές:

- **Υποδομή:** Από τοπικό *MicroK8s* στο μηχάνημα ανάπτυξης, σε εικονική μηχανή στο *Azure*, με την υποδομή να παρέχεται μέσω *Infrastructure as Code* (*OpenTofu*) αντί για χειροκίνητη εγκατάσταση.
- **Διαχείριση ρυθμίσεων:** Η ρύθμιση του συστήματος αυτοματοποιείται πλήρως μέσω *Ansible*, εξαλείφοντας τα χειροκίνητα βήματα εγκατάστασης που απαιτούνταν στο τοπικό περιβάλλον.
- **Διαχείριση secrets:** Τα ευαίσθητα διαπιστευτήρια (*MongoDB URI*, κλειδιά *MinIO*, διαπιστευτήρια *RabbitMQ*) μεταφέρονται από *hardcoded* ή τοπικές μεταβλητές περιβάλλοντος σε κεντρική, ασφαλή διαχείριση μέσω *HashiCorp Vault*.
- **Παράδοση κώδικα:** Η ανάπτυξη νέων εκδόσεων μεταβαίνει από χειροκίνητο *re-build/redeploy* σε πλήρως αυτοματοποιημένο *GitOps pipeline* (*Jenkins* για *continuous integration*, *ArgoCD* για *continuous deployment*), με κάθε *commit* να οδηγεί αυτόματα σε ενημέρωση του συστήματος.
- **Επεξεργασία εικόνων:** Η δημιουργία *thumbnails* προστίθεται ως ανεξάρτητη *serverless* συνάρτηση μέσω *Knative*, η οποία κλιμακώνεται αυτόματα ανάλογα με τη ζήτηση, αντί να αποτελεί μέρος του μονολιθικού *backend*.

Σε κάθε περίπτωση, το *application layer* (*frontend*, *backend*, *RabbitMQ*, *MinIO*, *Node-Red*, *Thingsboard*) παραμένει αμετάβλητο σε επίπεδο επιχειρηματικής λογικής — η μετάβαση αυτή δεν αλλάζει τι κάνει η εφαρμογή, αλλά πώς αναπτύσσεται, ρυθμίζεται, και λειτουργεί. Η παρακάτω εικόνα 2.1 συνοψίζει οπτικά αυτή τη μετάβαση, αντιπαραθέτοντας το *current state* με το *target state* της αρχιτεκτονικής.



Εικόνα 2.1: Εξέλιξη: από το *MicroK8s* τοπικής εγκατάστασης (*current state*) στο πλήρως αυτοματοποιημένο *cloud-native* περιβάλλον στο *Azure* (*target state*)

2.3 Ανάλυση *Microservices*

Το σύστημα αποτελείται από διακριτές υπηρεσίες, καθεμία με συγκεκριμένη ευθύνη, που επικοινωνούν μεταξύ τους μέσω καθορισμένων διεπαφών (*API* κλήσεις, μηνύματα ουράς, ή *events*). Η *frontend* υπηρεσία είναι υπεύθυνη αποκλειστικά για τη διεπαφή χρήστη, χωρίς να διατηρεί λογική επιχειρηματικού επιπέδου. Η *backend* υπηρεσία συγκεντρώνει τη βασική επιχειρηματική λογική: επεξεργασία αιτημάτων, έλεγχος *idempotency*, και συντονισμό των κλήσεων προς τις υπόλοιπες υπηρεσίες αποθήκευσης και ειδοποίησης.

Η αποθήκευση δεδομένων είναι διαχωρισμένη ανά τύπο περιεχομένου: τα δομημένα δεδομένα (στοιχεία προϊόντων, χρηστών) αποθηκεύονται στη *MongoDB*, ενώ τα μη-δομημένα δεδομένα μεγάλου μεγέθους (εικόνες προϊόντων) αποθηκεύονται στο *MinIO*, ως αντικείμενα (*object storage*). Ο διαχωρισμός αυτός επιτρέπει στις δύο υπηρεσίες να κλιμακώνονται ανεξάρτητα, ανάλογα με τη φύση του φόρτου που δέχονται.

Η επικοινωνία μεταξύ *backend* και των ασύγχρονων καταναλωτών (*Node-RED*, *Things-Board*) δεν γίνεται απευθείας, αλλά μέσω της ουράς μηνυμάτων *RabbitMQ*, η οποία αποσυνδέει χρονικά το *backend* από τους *consumers*: το *backend* δεν χρειάζεται να γνωρίζει πόσοι ή ποιοι καταναλωτές υπάρχουν, ούτε να περιμένει την ολοκλήρωση της επεξεργασίας τους για να απαντήσει στον χρήστη.

2.4 Τεχνολογικά κριτήρια επιλογής

Πέρα από τον ρόλο κάθε υπηρεσίας, αξίζει να σταθούμε στο γιατί επιλέχθηκε η συγκεκριμένη τεχνολογία για κάθε ρόλο, έναντι των πιο διαδεδομένων εναλλακτικών της αγοράς. Οι επιλογές αυτές δεν ήταν αυθαίρετες, αλλά βασίστηκαν σε κριτήρια καταλληλότητας για το μέγεθος και τη φύση του σεναρίου, αποφεύγοντας πολυπλοκότητα που δεν δικαιολογείται στην παρούσα κλίμακα.

Για την ασύγχρονη ανταλλαγή μηνυμάτων επιλέχθηκε το *RabbitMQ* αντί εναλλακτικών όπως το *Apache Kafka*: το *RabbitMQ* προσφέρει απλούστερο μοντέλο ουράς μηνυμάτων (*message queue*), κατάλληλο για τον όγκο και τη συχνότητα μηνυμάτων του σεναρίου, χωρίς την πολυπλοκότητα διαχείρισης *partitions* και *consumer groups* που συνεπάγεται το *Kafka*.

Για την αποθήκευση αρχείων επιλέχθηκε το *MinIO* ως αυτο-φιλοξενούμενη (*self-hosted*), συμβατή με *S3 API* λύση *object storage*, αντί απευθείας χρήσης *Amazon S3* ή *Azure Blob Storage*. Η επιλογή αυτή επιτρέπει στο σύστημα να παραμένει *cloud-agnostic* ως προς την αποθήκευση αντικειμένων, διατηρώντας πλήρη έλεγχο της υποδομής εντός του *Kubernetes cluster*, ενώ η συμβατότητα με το *S3 API* διασφαλίζει ότι ο κώδικας του *backend* θα παραμείνει λειτουργικός και σε περίπτωση μελλοντικής μετάβασης σε διαχειριζόμενη υπηρεσία *object storage*.

Ο πίνακας 1 συνοψίζει το σύνολο των τεχνολογιών υποδομής, αντιστοιχίζοντας κάθε υπηρεσία με τον τεχνολογικό της ρόλο.

Πίνακας 1: Application-level services and technology choices

Service	Technology	Role in infrastructure
Frontend	React	User interface
Backend	Node.js / Express	Business logic, API, design patterns
Messaging	RabbitMQ	Asynchronous communication (event bus)
Database	MongoDB Atlas	Managed NoSQL database service
Storage	MinIO	S3-compatible object storage
IoT logic	Node-RED	Flow-based environmental simulation
Visualization	ThingsBoard	IoT dashboards, real-time monitoring

2.5 Μόνιμη αποθήκευση δεδομένων (*Data Persistence*)

Τα *containers* είναι, εκ σχεδιασμού, εφήμερα: όταν ένα *pod* τερματίζεται ή αναδημιουργείται, οποιοδήποτε δεδομένο αποθηκευμένο στο εσωτερικό *filesystem* του *container* χάνεται. Για υπηρεσίες όπως το *MinIO*, το *RabbitMQ*, και το *Vault*, που απαιτούν διατήρηση δεδομένων ανεξάρτητα από τον κύκλο ζωής του *pod* που τις φιλοξενεί, το *Kubernetes* προσφέρει τον μηχανισμό *Persistent Volumes (PV)* και *Persistent Volume Claims (PVC)*.

Ένα *PV* αντιπροσωπεύει έναν πραγματικό αποθηκευτικό πόρο (στην περίπτωση αυτή, δίσκο στο *Azure*), ενώ ένα *PVC* αποτελεί το αίτημα ενός *pod* για χρήση συγκεκριμένης ποσότητας αποθηκευτικού χώρου, χωρίς να απαιτείται να γνωρίζει τις τεχνικές λεπτομέρειες του υποκείμενου *storage*. Η σύνδεση μεταξύ τους πραγματοποιείται αυτόματα από το *Kubernetes*, μέσω του προεπιλεγμένου *storage class (microk8s-hostpath)*, το οποίο δεσμεύει χώρο στον τοπικό δίσκο της εικονικής μηχανής.

Στο σύστημα, κάθε *stateful* υπηρεσία διαθέτει το δικό της *PVC*, ώστε τα δεδομένα της να επιβιώνουν ανεξάρτητα από επανεκκινήσεις: το *MinIO* διατηρεί τα αποθηκευμένα αρχεία, το *RabbitMQ* τη κατάσταση των ουρών, και το *Vault* τα κρυπτογραφημένα *secret* του, ακόμη και όταν τα αντίστοιχα *Pods* αναδημιουργούνται μετά από ενημέρωση ή επανεκκίνηση της εικονικής μηχανής.

Ενδεικτικά, ο ορισμός του *PVC* για το *RabbitMQ* έχει την ακόλουθη μορφή:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: rabbitmq-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

Listing 1: PersistentVolumeClaim για το RabbitMQ

Το *pod* του *RabbitMQ* αναφέρεται σε αυτό το *PVC* μέσω του συγκεκριμένου πεδίου `volumes.persistentVolumeClaim.claimName`, και προσαρτά τον αποθηκευτικό χώρο στον κατάλογο `/var/lib/rabbitmq`, όπου το *RabbitMQ* διατηρεί τα δεδομένα των ουρών του. Το αντίστοιχο μοτίβο ακολουθείται και για τα *PVCs* του *MinIO* και του *Vault*.

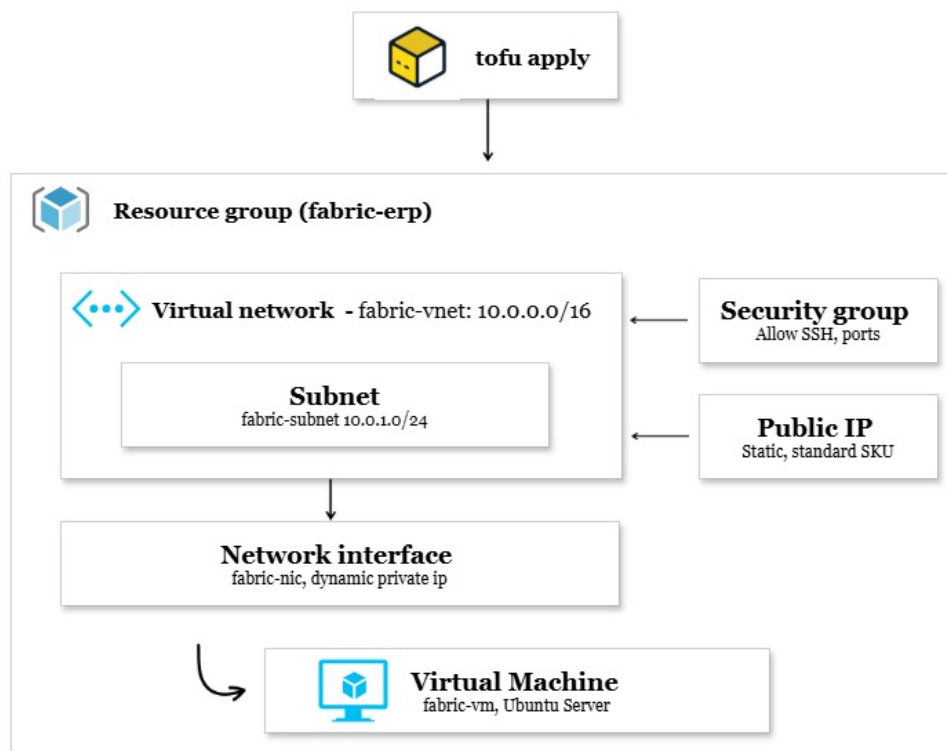
Η αρχιτεκτονική που παρουσιάστηκε στις προηγούμενες ενότητες αποτελεί το θεμέλιο πάνω στο οποίο χτίζονται οι υπόλοιπες πτυχές του συστήματος. Στα επόμενα κεφάλαια εξετάζεται το πώς αυτή η αρχιτεκτονική γίνεται **αυτοματοποιημένη** και ασφαλής (Κεφάλαιο 3), πώς ενσωματώνει σχεδιαστικά πρότυπα ανθεκτικότητας στον κώδικα (Κεφάλαιο 4), και πώς αξιολογείται η απόδοσή της μέσω πειραματικών μετρήσεων (Κεφάλαιο 5).

3 Κεφάλαιο 3: Αυτοματοποίηση υποδομών και ασφάλεια

Το παρόν κεφάλαιο παρουσιάζει τον τρόπο με τον οποίο η υποδομή που περιγράφηκε στο προηγούμενο κεφάλαιο δημιουργείται και ρυθμίζεται με αυτοματοποιημένο, αναπαραγωγίμο τρόπο: από τη δημιουργία της εικονικής μηχανής στο *Azure* μέσω *Infrastructure as Code* (*OpenTofu*), έως τη ρύθμιση του λογισμικού και των υπηρεσιών εντός αυτής μέσω *configuration management* (*Ansible*), και την κεντρική διαχείριση των ευαίσθητων δεδομένων του συστήματος (*HashiCorp Vault*).

3.1 *Infrastructure as Code* με *OpenTofu*

Η δημιουργία της εικονικής μηχανής στο *Microsoft Azure* δεν πραγματοποιήθηκε χειροκίνητα μέσω του *Azure Portal*, αλλά μέσω *OpenTofu* — ανοιχτού κώδικα *fork* του *Terraform* — ώστε ολόκληρη η υποδομή να ορίζεται δηλωτικά (*declarative*) σε αρχεία κώδικα, εκδοχοποιημένα στο σύστημα ελέγχου εκδόσεων. Η προσέγγιση αυτή (*Infrastructure as Code*) εξασφαλίζει ότι η υποδομή είναι αναπαραγωγίμη: σε περίπτωση καταστροφής ή ανάγκης δημιουργίας νέου περιβάλλοντος, η εκτέλεση των ίδιων αρχείων παράγει πανομοιότυπο αποτέλεσμα, χωρίς εξάρτηση από τη μνήμη ή τις χειροκίνητες ενέργειες κάποιου διαχειριστή.



Εικόνα 3.1: Ποή IaC: OpenTofu provisioning

Ο ορισμός της εικονικής μηχανής περιλαμβάνει το μέγεθος, το λειτουργικό σύστημα, και τον τρόπο πρόσβασης μέσω κλειδιού *SSH*:

```
resource "azurerm_linux_virtual_machine" "vm" {
  name                       = "${var.prefix}-vm"
  resource_group_name       = azurerm_resource_group.rg.name
  location                  = azurerm_resource_group.rg.location
  size                      = var.vm_size
  admin_username            = var.admin_username
  network_interface_ids     = [azurerm_network_interface.nic.id]

  admin_ssh_key {
    username   = var.admin_username
    public_key = file(pathexpand(var.ssh_public_key_path))
  }

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "StandardSSD_LRS"
    disk_size_gb        = 64
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts-gen2"
    version   = "latest"
  }
}
```

Listing 2: Ορισμός Azure Virtual Machine σε Terraform

Η πρόσβαση στο *virtual machine* γίνεται αποκλειστικά μέσω ζεύγους κλειδιών *SSH* (*admin_ssh_key*), χωρίς να ορίζεται κωδικός πρόσβασης — πρακτική που μειώνει σημαντικά την επιφάνεια επίθεσης σε σχέση με *password-based* πρόσβαση.

Η ασφάλεια σε επίπεδο δικτύου ρυθμίζεται μέσω *Network Security Group (NSG)*, όπου ορίζονται ρητά οι επιτρεπόμενες θύρες εισερχόμενης κίνησης:

```
security_rule {
  name                       = "K8sNodePorts"
  priority                  = 1004
  direction                 = "Inbound"
  access                    = "Allow"
  protocol                  = "Tcp"
  destination_port_range    = "30000-32767"
  source_address_prefix     = "*"
  destination_address_prefix = "*"
}
```

Listing 3: Security Rule για το Kubernetes NodePort

Το συγκεκριμένο εύρος θυρών (30000–32767) αντιστοιχεί στο προεπιλεγμένο εύρος που χρησιμοποιεί το *Kubernetes* για υπηρεσίες τύπου *NodePort* (*Jenkins*, *ArgoCD*, *MinIO*, κ.ά.), και ορίστηκε εκ των προτέρων στο επίπεδο της υποδομής, ώστε να μην απαιτείται μεταγενέστερη χειροκίνητη τροποποίηση του *firewall* κάθε φορά που εκτίθεται νέα υπηρεσία.

Τέλος, η υποδομή περιλαμβάνει έναν μηχανισμό αυτόματου τερματισμού της εικονικής μηχανής (*azurerm_dev_test_global_vm_shutdown_schedule*) στις 23:00 κάθε ημέρα, ως πρακτική περιορισμού κόστους (*FinOps*) για περιβάλλον ανάπτυξης που δεν απαιτεί συνεχή 24-ωρη λειτουργία. Παρακάτω φαίνεται το *tofu list* εφόσον είχε γίνει το *provisioning*:

```
ssn_command = ssh azureuser@20.188.60.198
● PS C:\Users\katsi\Documents\Projects\fabric-smart-warehouse\infrastructure> tofu state list
azurerm_dev_test_global_vm_shutdown_schedule.shutdown
azurerm_linux_virtual_machine.vm
azurerm_network_interface.nic
azurerm_network_interface_security_group_association.assoc
azurerm_network_security_group.nsg
azurerm_public_ip.pip
azurerm_resource_group.rg
azurerm_subnet.subnet
azurerm_virtual_network.vnet
● PS C:\Users\katsi\Documents\Projects\fabric-smart-warehouse\infrastructure> tofu output
public_ip = "20.188.60.198"
ssh_command = "ssh azureuser@20.188.60.198"
```

Εικόνα 3.2: Αποτέλεσμα του OpenTofu provisioning

3.2 Αυτοματοποιημένη εγκατάσταση με *Ansible*

Ενώ το *OpenTofu* είναι υπεύθυνο για τη δημιουργία της υποδομής (το «κέλυφος» — εικονική μηχανή, δίκτυο, *firewall*), η ρύθμιση του λογισμικού εντός αυτής της υποδομής αυτοματοποιείται μέσω *Ansible*. Η επιλογή δύο διαφορετικών εργαλείων για τα δύο αυτά επίπεδα — αντί ενός μόνο εργαλείου — αντικατοπτρίζει τον διαφορετικό χαρακτήρα των δύο εργασιών: το *OpenTofu* διαχειρίζεται πόρους cloud παρόχου μέσω *API* κλήσεων, ενώ το *Ansible* εκτελεί ακολουθίες εντολών εντός ήδη υπάρχοντος μηχανήματος.

Η αυτοματοποίηση είναι οργανωμένη σε επτά ανεξάρτητους *roles*, καθένας υπεύθυνος για μία διακριτή πτυχή του περιβάλλοντος: εγκατάσταση του *Docker* και του *MicroK8s* (*base-setup*), εγκατάσταση και ρύθμιση του *HashiCorp Vault* (*vault*), εγκατάσταση του *Prometheus* και *Grafana* (*monitoring*), εγκατάσταση του *Jenkins* (*jenkins*), εγκατάσταση του *ArgoCD* (*argocd*), ενεργοποίηση του *Knative* και εγκατάσταση του *MinIO Client* (*knative*), και εγκατάσταση εργαλείου εκτίμησης κόστους (*infracost*).

Κάθε εργασία (*task*) έχει σχεδιαστεί ώστε να είναι *idempotent*: η επανεκτέλεσή της δεν προκαλεί σφάλμα ή ανεπιθύμητη επανάληψη ενέργειας, αν το επιθυμητό αποτέλεσμα έχει ήδη επιτευχθεί. Ενδεικτικά, η εγκατάσταση του *MinIO Client* ελέγχει πρώτα την ύπαρξη του εκτελέσιμου αρχείου, και προχωράει στη λήψη του μόνο αν αυτό λείπει:

```

- name: Check if mc binary already exists
  stat:
    path: "{{ mc_install_path }}"
    register: mc_binary

- name: Download MinIO Client (mc) binary
  get_url:
    url: "{{ mc_download_url }}"
    dest: "{{ mc_install_path }}"
    mode: "0755"
  when: not mc_binary.stat.exists

```

Listing 4: Ansible task για την εγκατάσταση του MinIO Client

Η ιδιότητα αυτή είναι ιδιαίτερα σημαντική στο πλαίσιο της εργασίας, καθώς η εικονική μηχανή τερματίζεται και επανεκκινείται συχνά (μέσω `az vm deallocate / az vm start`, για λόγους περιορισμού κόστους), και το *playbook* πρέπει να μπορεί να εκτελεστεί ξανά με ασφάλεια μετά από κάθε επανεκκίνηση, χωρίς να απαιτείται διάκριση μεταξύ «πρώτης εκτέλεσης» και «επανεκτέλεσης». Στην παρακάτω εικόνα βλέπουμε το ansible μετά το run που έγινε συνολικά για όλα τα roles, αλλά και για το vault:

```

TASK [Install Vault via Helm (dev mode, exposed as NodePort)] *****
ok: [localhost]

TASK [Wait for Vault pod to be ready] *****
ok: [localhost]

TASK [Store application secrets in Vault] *****
changed: [localhost]

TASK [Enable Kubernetes auth method in Vault] *****
ok: [localhost]

TASK [Configure Kubernetes auth in Vault] *****
changed: [localhost]

TASK [Write Vault policy for backend] *****
changed: [localhost]

TASK [Create Vault Kubernetes role for backend] *****
changed: [localhost]

PLAY RECAP *****
localhost          : ok=17  changed=6  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

> azureuser@fabric-vm:~/fabric-smart-warehouse/ansible$

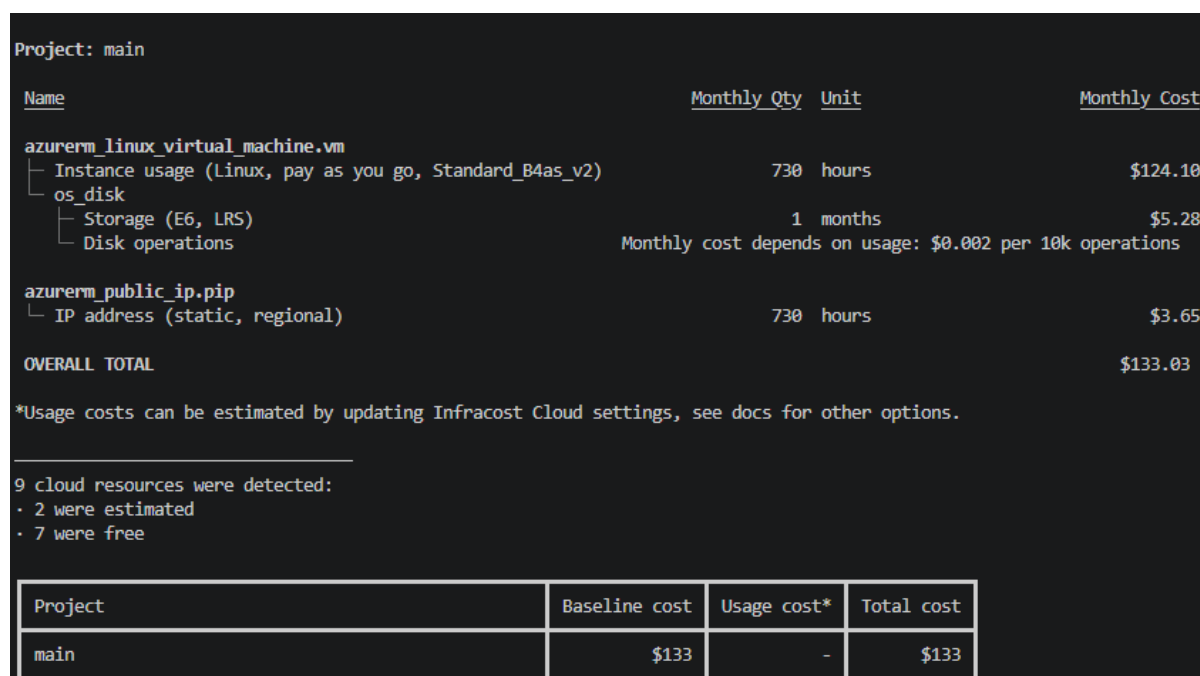
```

Εικόνα 3.3: Ansible: Run the playbook για κάθε role

3.3 Εκτίμηση κόστους υποδομής (*FinOps*): *Infracost*

Η αυτοματοποιημένη δημιουργία υποδομής μέσω *OpenTofu* διασφαλίζει αναπαραγωγικότητα, αλλά δεν δίνει από μόνη της ορατότητα στο οικονομικό αποτύπωμα της υποδομής. Για τον σκοπό αυτό ενσωματώθηκε το *Infracost*, ένα *CLI* εργαλείο που αναλύει στατικά τα αρχεία *.tf* και εκτιμά το μηνιαίο κόστος των πόρων που δηλώνονται, χωρίς να απαιτείται η πραγματική δημιουργία τους.

Η εκτέλεση `infracost breakdown -path infrastructure/` πάνω στους πόρους που περιγράφηκαν προηγουμένως, αποδίδει ανάλυση κόστους ανά πόρο: η εικονική μηχανή (*Standard_B4as_v2*, χρέωση ανά ώρα) αποτελεί το μεγαλύτερο μέρος του κόστους, με μικρότερη συμβολή από τον δίσκο λειτουργικού συστήματος και τη στατική *Public IP* (Εικόνα 3.4). Το σύνολο των 9 πόρων υποδομής που εντοπίστηκαν εκτιμάται σε **\$133.03** μηνιαίως.



```
Project: main
```

Name	Monthly Qty	Unit	Monthly Cost
azurelinux_virtual_machine.vm			
Instance usage (Linux, pay as you go, Standard_B4as_v2)	730	hours	\$124.10
os_disk			
Storage (E6, LRS)	1	months	\$5.28
Disk operations	Monthly cost depends on usage: \$0.002 per 10k operations		
azurepublic_ip.pip			
IP address (static, regional)	730	hours	\$3.65
OVERALL TOTAL			\$133.03

*Usage costs can be estimated by updating Infracost Cloud settings, see docs for other options.

9 cloud resources were detected:
 • 2 were estimated
 • 7 were free

Project	Baseline cost	Usage cost*	Total cost
main	\$133	-	\$133

Εικόνα 3.4: Ανάλυση κόστους υποδομής μέσω *Infracost*: εκτιμώμενο μηνιαίο κόστος \$133.03

Η εκτίμηση αυτή καλύπτει το επίπεδο της υποδομής όπως δηλώνεται στο *OpenTofu*, χωρίς να επεκτείνεται σε ανάλυση κόστους ανά μεμονωμένο *pod* σε πραγματικό χρόνο — περιορισμός που αναφέρεται στις μελλοντικές επεκτάσεις της εργασίας.

3.4 Διαχείριση των *secrets* με *HashiCorp Vault*

Η διαχείριση ευαίσθητων δεδομένων — διαπιστευτήρια βάσης δεδομένων, κλειδιά πρόσβασης σε αποθηκευτικό χώρο, κωδικοί πρόσβασης διαχειριστή — αποτελεί κρίσιμη πτυχή κάθε σύγχρονης cloud-native εφαρμογής. Η εναλλακτική προσέγγιση, η αποθήκευση αυτών των τιμών απευθείας μέσα στα αρχεία ρύθμισης (*manifests*) ή στον πηγαίο κώδικα, εκθέτει τα μυστικά αυτά σε καθαρό κείμενο, ορατό σε όποιον έχει πρόσβαση στο αποθετήριο *Git* — ανεξάρτητα από το αν το αποθετήριο είναι δημόσιο ή ιδιωτικό.

Για τον λόγο αυτό, το σύστημα χρησιμοποιεί το *HashiCorp Vault* ως κεντρικό σημείο αποθήκευσης όλων των ευαίσθητων τιμών: διαπιστευτήρια σύνδεσης στη *MongoDB*, κλειδιά πρόσβασης στο *MinIO*, κωδικός διαχειριστή του *ArgoCD*, και διαπιστευτήρια του *RabbitMQ*. Το *Vault* χρησιμοποιεί μόνιμη αποθήκευση σε αρχείο (*file storage*), προστατευμένη μέσω μηχανισμού *Shamir seal* — η αποκρυπτογράφηση των δεδομένων απαιτεί την παροχή κλειδιού αποσφράγισης (*unseal key*) πριν το *Vault* μπορέσει να εξυπηρετήσει οποιοδήποτε αίτημα ανάγνωσης ή εγγραφής.

Η πρόσβαση των εφαρμογών στα μυστικά δεν γίνεται μέσω σταθερού *token*, αλλά μέσω του μηχανισμού *Kubernetes authentication* του *Vault*: κάθε *pod* αποδεικνύει την ταυτότητά του μέσω του δικού του *service account token*, και το *Vault* του παραχωρεί πρόσβαση μόνο στα μυστικά που επιτρέπει η αντίστοιχη πολιτική (*policy*) που έχει ανατεθεί στον ρόλο του.

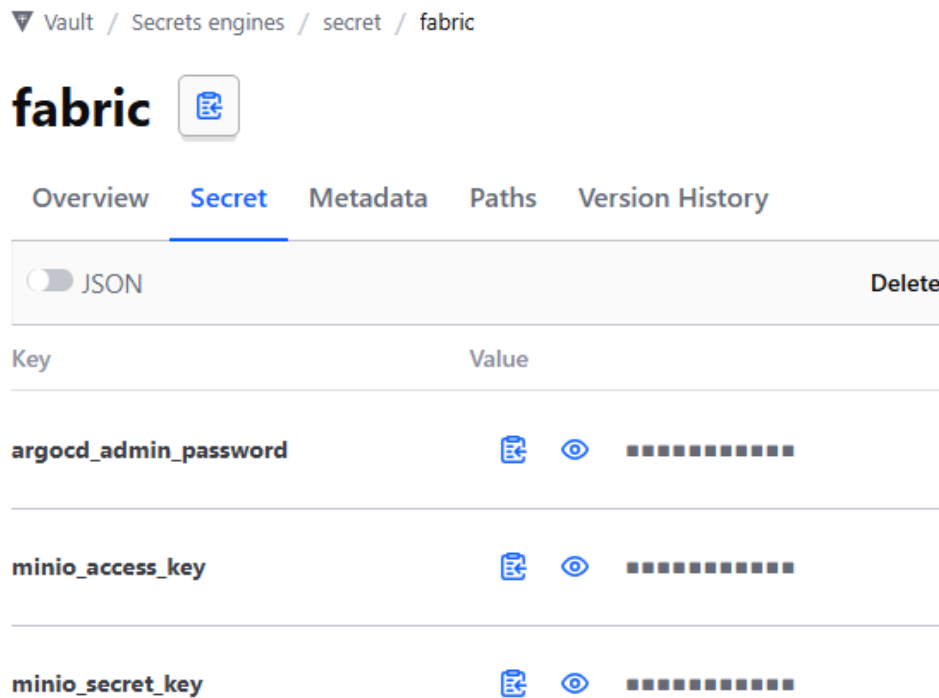
Η ενσωμάτωση αυτή πραγματοποιείται μέσω του *Vault Agent Injector*: το *pod* του *backend* φέρει συγκεκριμένες επισημειώσεις (*annotations*) που ενεργοποιούν αυτόματη εισαγωγή ενός δεύτερου, βοηθητικού *container* (*sidecar*) μέσα στο ίδιο *pod*. Αυτός ο *sidecar* αυθεντικοποιείται στο *Vault*, ανακτά τα μυστικά, και τα γράφει σε ένα αρχείο εντός κοινού, εφήμερου χώρου αποθήκευσης (*volume*) που μοιράζονται και τα δύο *containers* του *pod*. Ο κώδικας του *backend* διαβάζει αυτό το αρχείο κατά την εκκίνησή του, με ασφαλή *fallback* σε μεταβλητές περιβάλλοντος αν το αρχείο δεν υπάρχει:

```
let vaultSecrets = {};  
try {  
  const raw = fs.readFileSync("/vault/secrets/config", "utf8");  
  vaultSecrets = JSON.parse(raw);  
} catch (err) {  
  console.log("Vault secrets not found, falling back to  
    environment variables");  
}  
  
const MONGO_URI = vaultSecrets.mongo_uri || process.env.MONGO_URI;
```

Listing 5: Φόρτωση μυστικών από το Vault με fall-back

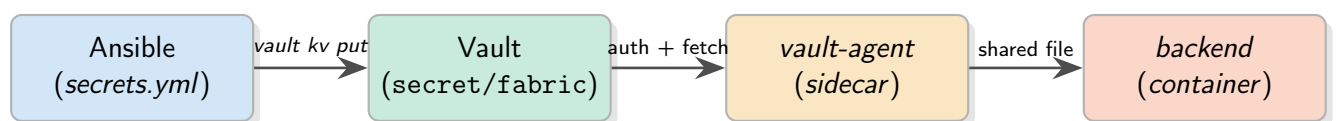
Το αρχείο */vault/secrets/config* που διαβάζει ο παραπάνω κώδικας δεν δημιουργείται από το ίδιο το *backend*, αλλά από τον *Vault Agent Injector*: ένα *sidecar container* που τρέχει στο ίδιο *pod*, αυθεντικοποιείται αυτόματα στο *Vault* μέσω του *Kubernetes Service Account* του *pod*, ανακτά τα μυστικά από τη διαδρομή (*path*) *secret/fabric*, και τα γράφει τοπικά σε αυτό το αρχείο πριν ακόμα ξεκινήσει η εφαρμογή. Το *try/catch* εξασφαλίζει ότι η εφαρμογή παραμένει λειτουργική και σε περιβάλλον όπου ο *Vault* δεν είναι διαθέσιμος (π.χ. τοπική ανάπτυξη), επιστρέφοντας στις παραδοσιακές μεταβλητές περιβάλλοντος.

Τα μυστικά αυτά — η διεύθυνση σύνδεσης στη *MongoDB*, τα κλειδιά πρόσβασης στο *MinIO*, και τα διαπιστευτήρια του *RabbitMQ* — είναι ορατά μέσω του *web UI* του *Vault*, όπως φαίνεται στην [Figure 3.5](#), αποθηκευμένα κεντρικά αντί να βρίσκονται *hardcoded* ή διασκορπισμένα σε πολλαπλά αρχεία ρυθμίσεων.



Εικόνα 3.5: HashiCorp Vault: αποθηκευμένα μυστικά εφαρμογής στο *path secret/fabric*

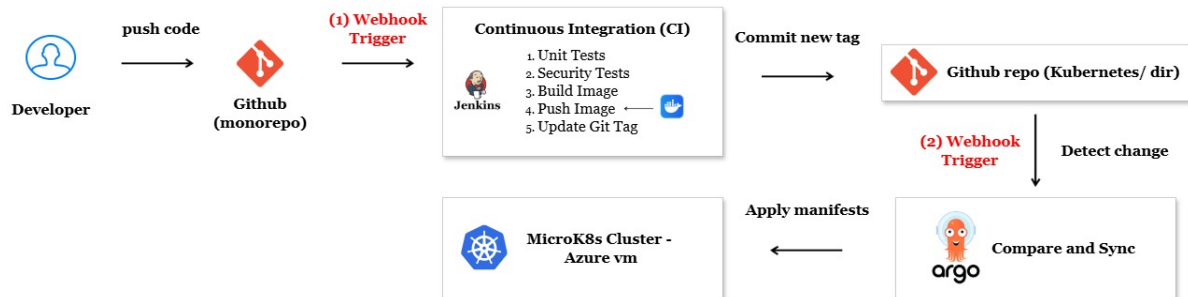
Η ροή αυτή απεικονίζεται στο σχήμα παρακάτω σχήμα: το *Ansible* γράφει τις τιμές στο *Vault* κατά την αρχική ρύθμιση, ο *sidecar* τις ανακτά σε κάθε εκκίνηση του *pod*, και το *backend* τις διαβάζει μέσω κοινού αρχείου.



Η αυτοματοποίηση της υποδομής και η ασφαλής διαχείριση των μυστικών της εφαρμογής, όπως παρουσιάστηκαν στο παρόν κεφάλαιο, αποτελούν το θεμέλιο πάνω στο οποίο λειτουργούν οι διαδικασίες συνεχούς ολοκλήρωσης και ανάπτυξης που εξετάζονται στο επόμενο κεφάλαιο: χωρίς αναπαραγώγιμη υποδομή και ασφαλή διαχείριση διαπιστευτηρίων, η αυτοματοποιημένη ανάπτυξη νέου κώδικα θα ήταν τόσο επικίνδυνη όσο και η χειροκίνητη.

4 Κεφάλαιο 4: *GitOps* και *CI/CD Pipelines*

Το παρόν κεφάλαιο περιγράφει πώς μια αλλαγή στον κώδικα της εφαρμογής μεταφράζεται αυτόματα σε αλλαγή στο τρέχον σύστημα: από τη δομή του αποθετηρίου *Git*, μέσω της συνεχούς ενσωμάτωσης (*Continuous Integration*) με το *Jenkins*, έως τη συνεχή παράδοση (*Continuous Deployment*) με το *ArgoCD* — χωρίς να απαιτείται καμία χειροκίνητη ενέργεια μετά την αρχική υποβολή του κώδικα.



Εικόνα 4.1: GitOps pipeline: από το git push στο cluster μέσω Jenkins και ArgoCD

4.1 Δομή του *Git Monorepo*

Ο πηγαίος κώδικας του συστήματος είναι οργανωμένος σε ένα ενιαίο αποθετήριο (*monorepo*), όπου κάθε επιμέρους συστατικό — *backend*, *frontend*, *serverless* συνάρτηση, αρχεία υποδομής — διατηρείται στον δικό του υποκατάλογο, με το δικό του *Dockerfile* όπου απαιτείται. Η επιλογή *monorepo* αντί πολλαπλών, ανεξάρτητων αποθετηρίων διευκολύνει τη συνολική παρακολούθηση των αλλαγών και απλοποιεί τη ρύθμιση τόσο του *Jenkins* όσο και του *ArgoCD*, καθώς και τα δύο εργαλεία αναφέρονται σε ένα και μόνο αποθετήριο.

Ο κατάλογος *kubernetes/* συγκεντρώνει το σύνολο των *manifests* ανάπτυξης (*Deployments*, *Services*, *PersistentVolumeClaims*) για όλα τα συστατικά του συστήματος, και αποτελεί το σημείο που παρακολουθεί το *ArgoCD*: οποιαδήποτε αλλαγή εντός αυτού του καταλόγου αποτελεί *desired state* για το *cluster*. Από τη στιγμή που το *Kustomize* εισήχθη ως επιπλέον επίπεδο οργάνωσης αυτού του καταλόγου, ένα αρχείο *kustomization.yaml* απαριθμεί ρητά το σύνολο των *manifests* που πρέπει να συναρμολογηθούν, χωρίς να μεταβάλλεται το περιεχόμενο των επιμέρους αρχείων.

4.2 *Continuous Integration - (CI)* με *Jenkins*

Το *Jenkins* αναλαμβάνει τον ρόλο της συνεχούς ενσωμάτωσης: από τη στιγμή που ο κώδικας *push*-άρεται στο αποθετήριο, το *pipeline* εκτελεί αυτόματα μια σειρά ελέγχων και ενεργειών — δοκιμές, ελέγχους ασφάλειας, δημιουργία και δημοσίευση *image* — και καταλήγει σε ενημέρωση του *desired state* που παρακολουθεί το *ArgoCD*. Οι επόμενες ενότητες παρουσιάζουν αυτή τη ροή ως μία αλυσίδα: πώς ενεργοποιείται αυτόματα το *pipeline*, ποια στάδια εκτελούνται κατά τον έλεγχο και τη δημιουργία της εικόνας, και πώς η ενημέρωση του *manifest* κλείνει τον κύκλο.

4.2.1 Αυτόματη ενεργοποίηση μέσω *webhook* και επιλεκτική εκτέλεση

Η αρχική ρύθμιση του *pipeline* απαιτούσε χειροκίνητη εκκίνηση από τον χρήστη («*Build Now*») μετά από κάθε *push* κώδικα. Για την πλήρη αυτοματοποίηση αυτού του βήματος, ρυθμίστηκε *webhook* στο επίπεδο του αποθετηρίου *GitHub*: κάθε *push* στον κλάδο *main* αποστέλλει αυτόματα ειδοποίηση στον *server* του *Jenkins*, η οποία ενεργοποιεί νέα εκτέλεση του *pipeline* χωρίς καμία ανθρώπινη παρέμβαση.

Καθώς το αποθετήριο είναι οργανωμένο ως *monorepo*, μια αλλαγή σε οποιοδήποτε αρχείο — ανεξάρτητα από το αν αφορά τον κώδικα της εφαρμογής ή απλά τεκμηρίωση — θα ενεργοποιούσε, χωρίς πρόσθετο έλεγχο, πλήρη εκτέλεση *build* και δημοσίευσης νέας έκδοσης (*image*) στο *Docker Hub*, ανεξάρτητα από το αν ο κώδικας της εφαρμογής είχε όντως μεταβληθεί. Για την αποφυγή αυτής της άσκοπης επανάληψης, κάθε στάδιο (*stage*) του *pipeline* που αφορά το *backend* ή το *frontend* συνδέεται με συνθήκη τύπου `when { changeset }`, η οποία ελέγχει αν το *Git changeset* της τρέχουσας εκτέλεσης περιλαμβάνει αρχεία εντός του αντίστοιχου καταλόγου:

```
stage('Backend: Build Docker Image') {
    when {
        changeset "backend/**"
    }
    steps {
        dir('backend') {
            sh "docker build -t ${BACKEND_IMAGE_NAME}:${IMAGE_TAG} ."
        }
    }
}
```

Listing 6: Jenkins Pipeline stage για το build του Backend

Με τον τρόπο αυτό, μια αλλαγή αποκλειστικά στο *backend/* ενεργοποιεί μόνο τα στάδια του *backend*, μια αλλαγή αποκλειστικά στο *frontend/* ενεργοποιεί μόνο τα αντίστοιχα στάδια του *frontend*, και μια αλλαγή εκτός αμφοτέρων (π.χ. σε αρχεία υποδομής ή τεκμηρίωσης) παραλείπει εντελώς τα στάδια *build* και *push*, χωρίς να δημιουργείται νέα, αχρείαστη έκδοση *image*.

4.2.2 Το *Jenkinsfile* και η διαδικασία *Build & Test*

Το *pipeline* εκτελείται μέσα σε προσωρινό *pod*, δημιουργημένο δυναμικά εντός του ίδιου *Kubernetes cluster* (*Jenkins Kubernetes plugin*), με ξεχωριστά *containers* για τις εργασίες *Node.js* και *Docker*. Μετά την επιβεβαίωση ότι η αλλαγή αφορά τον κατάλογο *backend/*, εκτελούνται διαδοχικά: μοναδιαίοι έλεγχοι (*unit tests*) μέσω *Jest*, και έλεγχος ασφάλειας εξαρτήσεων (*npm audit*) με όριο αποδοχής σε κρίσιμες (*critical*) ευπάθειες.

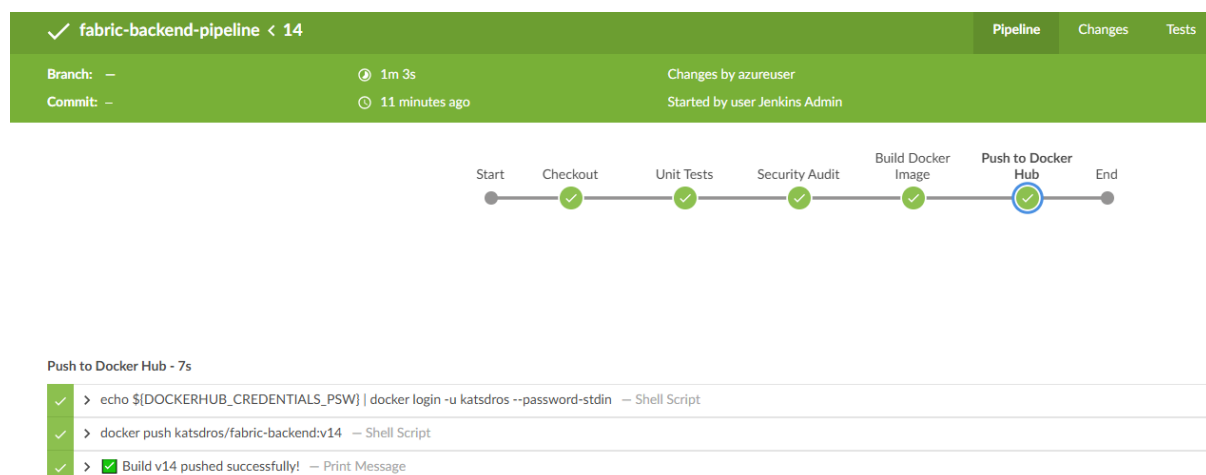
```

stage('Backend: Unit Tests') {
    when { changeset "backend/**" }
    steps {
        container('node') {
            dir('backend') {
                sh 'npm install && npm test'
            }
        }
    }
}

```

Listing 7: Jenkins Pipeline stage για το εκτέλεση Unit Tests

Μόνο εφόσον αμφότεροι οι έλεγχοι ολοκληρωθούν επιτυχώς, το *pipeline* προχωράει στο επόμενο στάδιο, της δημιουργίας του *container image*. Η αποτυχία οποιουδήποτε ελέγχου διακόπτει αυτόματα την εκτέλεση, αποτρέποντας τη δημοσίευση μη ελεγμένου κώδικα. Η εικόνα 4.2 δείχνει μια ολοκληρωμένη, επιτυχημένη εκτέλεση του *pipeline*, με το σύνολο των σταδίων — δοκιμές, έλεγχο ασφάλειας, δημιουργία *image* — να εμφανίζεται με πράσινη ένδειξη επιτυχίας.

Εικόνα 4.2: Επιτυχής εκτέλεση του *Jenkins pipeline*

Η επιτυχής δημιουργία του *image*, όπως φαίνεται παραπάνω, αποτελεί προαπαιτούμενο για το επόμενο στάδιο: η εικόνα δεν έχει κανένα νόημα αν παραμείνει τοπικά αποθηκευμένη μέσα στο *pod* εκτέλεσης του *Jenkins* — πρέπει να δημοσιευτεί σε σημείο όπου το *cluster* μπορεί να την ανακτήσει κατά την ανάπτυξη (*deployment*).

4.2.3 Push των container images στο Docker Hub

Κάθε επιτυχημένη εκτέλεση παράγει νέα έκδοση (*image*) του *backend*, με ετικέτα (*tag*) που αντιστοιχεί στον αριθμό εκτέλεσης του *Jenkins* (`v${BUILD_NUMBER}`), εξασφαλίζοντας ότι κάθε έκδοση είναι μοναδική και ιχνηλατήσιμη. Η εικόνα δημιουργείται και δημοσιεύεται στο *Docker Hub* μέσω των ακόλουθων σταδίων:

```
stage('Backend: Push to Docker Hub') {
    when { changeset "backend/**" }
    steps {
        withCredentials([usernamePassword(
            credentialsId: 'dockerhub-credentials',
            usernameVariable: 'DOCKERHUB_USR',
            passwordVariable: 'DOCKERHUB_PSW')]) {
            sh "echo ${DOCKERHUB_PSW} | docker login -u ${
                DOCKERHUB_USR} --password-stdin"
            sh "docker push ${BACKEND_IMAGE_NAME}:${IMAGE_TAG}"
        }
    }
}
```

Listing 8: Jenkins Pipeline stage για το push της εικόνας στο Docker Hub

Τα διαπιστευτήρια πρόσβασης στο *Docker Hub* δεν εμφανίζονται πουθενά σε καθαρό κείμενο: είναι αποθηκευμένα στο *Jenkins Credentials Store*, και εισάγονται στο περιβάλλον εκτέλεσης αποκλειστικά μέσω του *block withCredentials*, το οποίο τα αποκρύπτει αυτόματα από τα αρχεία καταγραφής (*logs*) του *pipeline*.

Αμέσως μετά τη δημοσίευση, ένα τελικό στάδιο ενημερώνει αυτόματα το *manifest* ανάπτυξης (*kubernetes/backend-deployment.yaml*) με το νέο *tag*, και πραγματοποιεί *commit* και *push* της αλλαγής πίσω στο ίδιο αποθετήριο — κλείνοντας έτσι τον κύκλο μεταξύ της δημιουργίας νέας έκδοσης και της ενημέρωσης του επιθυμητού (*desired*) *state* που παρακολουθεί το *ArgoCD*. Για την αποφυγή ατέρμονου βρόχου — όπου το αυτόματο *commit* θα ενεργοποιούσε ξανά το ίδιο το *pipeline* — το μήνυμα του *commit* φέρει την ετικέτα `[skip-jenkins]`, η οποία ελέγχεται σε αρχικό στάδιο (*guard*) πριν την εκτέλεση οποιουδήποτε άλλου σταδίου.

4.3 Continuous Delivery & Deployment - (CD) με ArgoCD

Ενώ το *Jenkins* είναι υπεύθυνο για τη δημιουργία και δημοσίευση νέων εκδόσεων (*images*), η εφαρμογή τους στο *cluster* δεν πραγματοποιείται απευθείας από το *Jenkins*. Αντί αυτού, ακολουθείται η φιλοσοφία *GitOps*: το *Git* αποθετήριο αποτελεί τη **μοναδική πηγή αλήθειας** (*single source of truth*) για την επιθυμητή κατάσταση του *cluster*, και ένας ξεχωριστός ελεγκτής — το *ArgoCD* — είναι υπεύθυνος να διασφαλίζει ότι η πραγματική κατάσταση του *cluster* συγκλίνει συνεχώς προς αυτή την επιθυμητή κατάσταση.

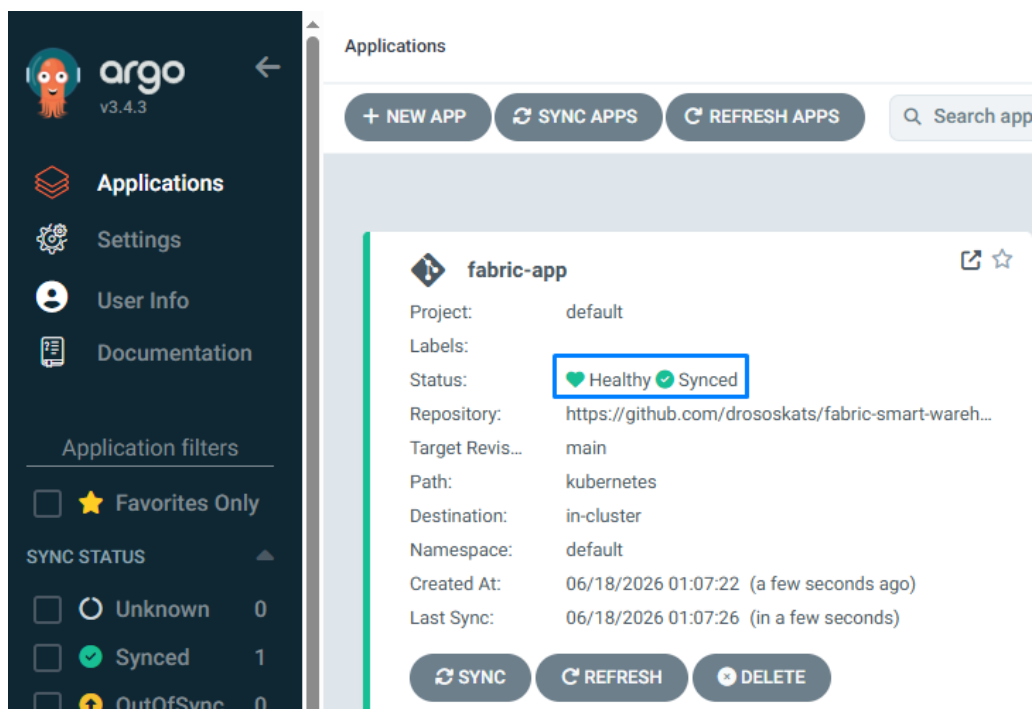
Το αντικείμενο *Application* του *ArgoCD* ορίζει το αποθετήριο και τον κατάλογο που παρακολουθείται (*kubernetes/*), καθώς και την πολιτική συγχρονισμού:

```
spec:
  source:
    repoURL: https://github.com/drososkats/fabric-smart-warehouse.git
    targetRevision: main
    path: kubernetes
  syncPolicy:
    automated:
      enabled: true
      prune: true
      selfHeal: true
```

Listing 9: ArgoCD Application manifest για το συγχρονισμό του repository

Η ρύθμιση `selfHeal: true` εξασφαλίζει ότι, αν κάποιος τροποποιήσει χειροκίνητα έναν πόρο απευθείας στο *cluster* (π.χ. μέσω `kubectl edit`), το *ArgoCD* θα επαναφέρει αυτόματα τον πόρο στην κατάσταση που ορίζει το *Git*, αποτρέποντας *drift* μεταξύ των δύο. Η ρύθμιση `prune: true` διασφαλίζει ότι πόροι που αφαιρούνται από το *Git* αφαιρούνται αντίστοιχα και από το *cluster*.

Από προεπιλογή, το *ArgoCD* ανιχνεύει αλλαγές στο αποθετήριο μέσω περιοδικής ανίχνευσης (*polling*), με διάστημα μερικών λεπτών. Για τη μείωση αυτής της καθυστέρησης, ρυθμίστηκε επιπλέον *webhook* — κατά το ίδιο πρότυπο με το *Jenkins* — ώστε κάθε *push* στο αποθετήριο να ενεργοποιεί άμεση επαναξιολόγηση (*refresh*) από το *ArgoCD*, χωρίς αναμονή του επόμενου κύκλου *polling*.



Εικόνα 4.3: *ArgoCD*: εφαρμογή σε κατάσταση *Healthy* και *Synced* μετά από αυτόματο συγχρονισμό

Η ροή που περιγράφηκε — από το *push* κώδικα, μέσω αυτόματου ελέγχου και δημιουργίας νέας έκδοσης στο *Jenkins*, έως τον αυτόματο συγχρονισμό του *cluster* από το *ArgoCD* — συνθέτει έναν πλήρως αυτοματοποιημένο κύκλο, χωρίς καμία χειροκίνητη παρέμβαση μετά την αρχική υποβολή κώδικα. Το επόμενο κεφάλαιο εξετάζει τι ακριβώς περιέχει ο κώδικας που διανύει αυτόν τον κύκλο — τα σχεδιαστικά πρότυπα ανθεκτικότητας που τον καθιστούν ανθεκτικό σε σφάλματα και κατάλληλο για κλιμάκωση.

5 Κεφάλαιο 5: Μοτίβα σχεδιασμού και ανθεκτικότητα εφαρμογής

Η αυτοματοποίηση της ανάπτυξης που περιγράφηκε στα προηγούμενα κεφάλαια διασφαλίζει ότι ο κώδικας φτάνει στο *cluster* με αξιόπιστο και ελεγμένο τρόπο. Το παρόν κεφάλαιο εξετάζει τι περιέχει αυτός ο κώδικας: τα σχεδιαστικά πρότυπα (*design patterns*) που τον καθιστούν ανθεκτικό σε αναμενόμενες αστοχίες — αποτυχημένες συνδέσεις, διπλά αιτήματα, μεταφορά μεγάλου όγκου δεδομένων — χωρίς να απαιτείται η παρέμβαση διαχειριστή για κάθε τέτοιο σφάλμα. Σκοπός των επόμενων ενοτήτων είναι να παρουσιάσουν τον κώδικα της εφαρμογής που την κάνει «άθραυστη» (*resilient*): ικανή να συνεχίσει να λειτουργεί ορθά παρά τις αναπόφευκτες, μερικές αστοχίες ενός κατακευματισμένου συστήματος.

5.1 *Claim Check Pattern*

Οι ουρές μηνυμάτων, όπως το *RabbitMQ*, είναι σχεδιασμένες για τη μεταφορά μικρών, ελαφρών μηνυμάτων — όχι μεγάλων αρχείων. Η απευθείας ενσωμάτωση μιας εικόνας μέσα στο μήνυμα προς το *RabbitMQ* θα επιβάρυνε σημαντικά την ουρά, θα μείωνε την ταχύτητα επεξεργασίας, και θα αύξανε τον κίνδυνο αποτυχίας μεταφοράς για μεγάλα αρχεία.

Το μοτίβο *Claim Check* αντιμετωπίζει αυτό το πρόβλημα διαχωρίζοντας τη μεταφορά του περιεχομένου από τη μεταφορά της ειδοποίησης: το αρχείο αποθηκεύεται απευθείας σε εξειδικευμένο αποθηκευτικό χώρο (*MinIO*), και μόνο μια ελαφριά αναφορά — το *URL* του αποθηκευμένου αντικειμένου — περνάει μέσα στο μήνυμα προς το *RabbitMQ* και αποθηκεύεται στη βάση δεδομένων:

```
async function uploadAndGetClaimCheck(minioClient, bucket, file,
  prefix, vmIp) {
  const fileName = `${prefix}-${Date.now()}-${file.originalname}`;
  await minioClient.fPutObject(bucket, fileName, file.path);
  fs.unlinkSync(file.path);
  return `http://${vmIp}:9000/${bucket}/${fileName}`;
}
```

Listing 10: JavaScript συνάρτηση για ανέβασμα αρχείου στο MinIO και επιστροφή του claim check

Η συνάρτηση αυτή χρησιμοποιείται τόσο για την εικόνα όσο και για το παραστατικό (*invoice*) ενός προϊόντος, επιστρέφοντας σε κάθε περίπτωση μόνο το *URL* αναφοράς — την «απόδειξη παραλαβής» (*claim check*) — που χρειάζεται οποιοδήποτε σύστημα κατάντη (*downstream*) για να ανακτήσει το πραγματικό αρχείο, όταν και αν αυτό χρειαστεί.

5.2 *Retry Pattern*

Κατά την εκκίνηση του *backend container*, η σύνδεση με την εξωτερική βάση δεδομένων (*MongoDB Atlas*) μπορεί να αποτύχει προσωρινά — για παράδειγμα, λόγω καθυστέρησης στην ετοιμότητα του δικτύου του *cluster*, ή στιγμιαίας μη διαθεσιμότητας της υπηρεσίας. Χωρίς μηχανισμό αντιμετώπισης, μια τέτοια προσωρινή αποτυχία θα οδηγούσε σε πλήρη διακοπή του *pod*, απαιτώντας χειροκίνητη επανεκκίνηση.

Το μοτίβο *Retry* αντιμετωπίζει αυτή την περίπτωση επαναλαμβάνοντας την προσπάθεια σύνδεσης πεπερασμένο αριθμό φορές, με σταθερό χρονικό διάστημα αναμονής μεταξύ των προσπαθειών, πριν αναγνωρίσει οριστική αποτυχία:

```
async function retryWithDelay(fn, { retries = 3, delayMs = 2000,
  label } = {}) {
  for (let attempt = 1; attempt <= retries; attempt++) {
    try {
      return await fn();
    } catch (err) {
      if (attempt < retries) {
        await new Promise(resolve => setTimeout(resolve, delayMs))
          ;
      } else {
        throw err;
      }
    }
  }
}
```

Listing 11: JavaScript συνάρτηση `retryWithDelay` για ανθεκτικότητα στο σύστημα

Η συνάρτηση είναι σχεδιασμένη ως γενικευμένο, επαναχρησιμοποιήσιμο εργαλείο (δέχεται οποιαδήποτε ασύγχρονη λειτουργία ως παράμετρο), και όχι ως λογική αποκλειστικά συνδεδεμένη με τη *MongoDB*. Στο σύστημα, χρησιμοποιείται για τη σύνδεση με τη βάση δεδομένων κατά την εκκίνηση, με τρεις προσπάθειες και διάστημα δύο δευτερολέπτων μεταξύ τους.

5.3 *Idempotency Pattern*

Σε διεπαφές χρήστη που εμπλέκουν υποβολή φόρμας με μεταφόρτωση αρχείου, είναι σύνηθες φαινόμενο η αθέλητη διπλή υποβολή — για παράδειγμα, λόγω καθυστέρησης απόκρισης του διακομιστή και επανειλημμένου πατήματος του κουμπιού υποβολής από τον χρήστη. Χωρίς προστασία, κάθε υποβολή θα δημιουργούσε ξεχωριστή εγγραφή στη βάση δεδομένων, παράγοντας διπλά προϊόντα.

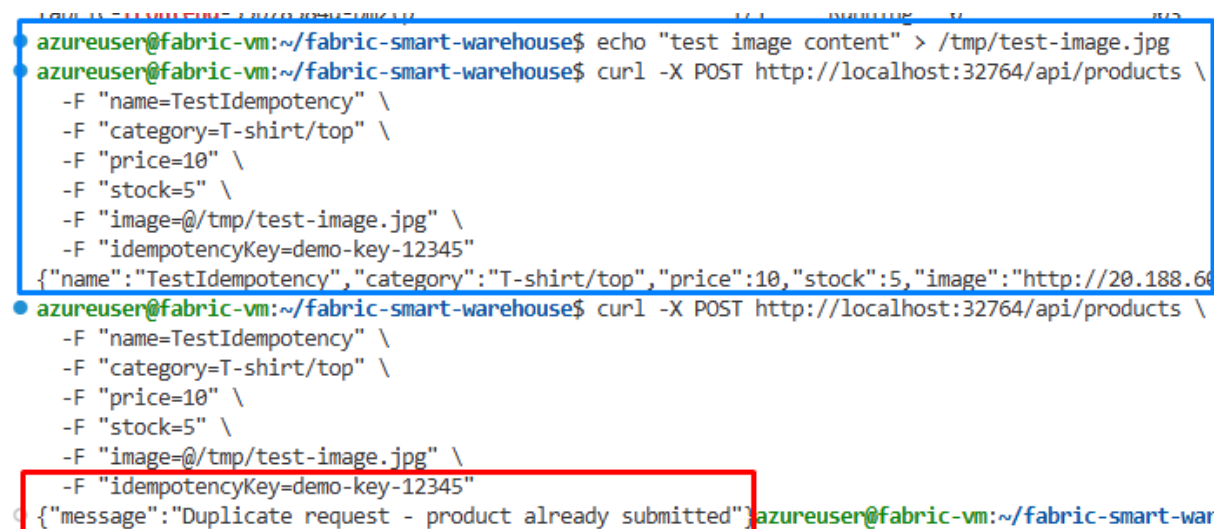
Το μοτίβο *Idempotency* αντιμετωπίζει αυτό το πρόβλημα απαιτώντας από τον *client* να αποστέλλει ένα μοναδικό κλειδί (*idempotency key*) μαζί με κάθε αίτημα. Ο *server* διατηρεί τα κλειδιά που έχει ήδη επεξεργαστεί, και απορρίπτει άμεσα οποιοδήποτε αίτημα με κλειδί που έχει ξαναχρησιμοποιηθεί:

```
const idempotencyKey = req.body.idempotencyKey;
const { duplicate } = checkIdempotency(idempotencyKey);
if (duplicate) {
  return res.status(409).json({ message: "Duplicate request -
    product already submitted" });
}
```

Listing 12: Έλεγχος Idempotency για την αποφυγή διπλότυπων αιτημάτων

Η ορθότητα της υλοποίησης επιβεβαιώνεται μέσω αυτοματοποιημένων μοναδιαίων ελέγχων (*unit tests*), που εξετάζουν τρία σενάρια: αίτημα με νέο κλειδί (δεν πρέπει να αναγνωριστεί ως διπλό), αίτημα με ήδη χρησιμοποιημένο κλειδί (πρέπει να αναγνωριστεί ως διπλό), και αίτημα χωρίς κλειδί (δεν πρέπει ποτέ να αναγνωριστεί ως διπλό, καθώς δεν υπάρχει κλειδί προς σύγκριση).

Η συμπεριφορά αυτή επιβεβαιώνεται και σε πραγματικές συνθήκες, πέρα από το περιβάλλον των αυτοματοποιημένων ελέγχων: όπως φαίνεται στην [Figure 5.1](#), ένα πρώτο αίτημα POST με συγκεκριμένο *idempotency key* δημιουργεί επιτυχώς το προϊόν, ενώ ένα δεύτερο, πανομοιότυπο αίτημα με το ίδιο κλειδί απορρίπτεται άμεσα με κωδικό κατάστασης 409 Conflict, χωρίς να δημιουργηθεί δεύτερη εγγραφή στη βάση δεδομένων.



```
azureuser@fabric-vm:~/fabric-smart-warehouse$ echo "test image content" > /tmp/test-image.jpg
azureuser@fabric-vm:~/fabric-smart-warehouse$ curl -X POST http://localhost:32764/api/products \
-F "name=TestIdempotency" \
-F "category=T-shirt/top" \
-F "price=10" \
-F "stock=5" \
-F "image=@/tmp/test-image.jpg" \
-F "idempotencyKey=demo-key-12345"
{"name":"TestIdempotency","category":"T-shirt/top","price":10,"stock":5,"image":"http://20.188.64
azureuser@fabric-vm:~/fabric-smart-warehouse$ curl -X POST http://localhost:32764/api/products \
-F "name=TestIdempotency" \
-F "category=T-shirt/top" \
-F "price=10" \
-F "stock=5" \
-F "image=@/tmp/test-image.jpg" \
-F "idempotencyKey=demo-key-12345"
{"message":"Duplicate request - product already submitted"} azureuser@fabric-vm:~/fabric-smart-war
```

Εικόνα 5.1: Δοκιμή του *Idempotency Pattern* μέσω *curl*: το δεύτερο αίτημα με το ίδιο *idempotency key* απορρίπτεται με 409 Conflict

5.4 Stateless Pattern

Σε αντίθεση με τα τρία προηγούμενα μοτίβα, η στατελής (*stateless*) φύση του *backend* δεν εντοπίζεται σε ένα συγκεκριμένο τμήμα κώδικα, αλλά αποτελεί ιδιότητα της συνολικής αρχιτεκτονικής του: ο *server* δεν διατηρεί καμία πληροφορία συνεδρίας (*session*) στη μνήμη του μεταξύ διαδοχικών αιτημάτων. Η αυθεντικοποίηση του χρήστη βασίζεται σε *JSON Web Tokens (JWT)*, τα οποία μεταφέρουν την απαραίτητη πληροφορία ταυτότητας μέσα στο ίδιο το αίτημα, και κάθε δεδομένο που πρέπει να διατηρηθεί αποθηκεύεται μόνιμα στη *MongoDB*, όχι στη μνήμη του *process*.

Η ιδιότητα αυτή είναι προαπαιτούμενο για την οριζόντια κλιμάκωση (*horizontal scaling*) του *backend*: επειδή κανένα αντίτυπο (*replica*) δεν διατηρεί τοπική κατάσταση, οποιοδήποτε αντίτυπο μπορεί να εξυπηρετήσει οποιοδήποτε αίτημα, χωρίς ανάγκη «κολλώδους» σύνδεσης (*sticky session*) μεταξύ συγκεκριμένου χρήστη και συγκεκριμένου αντιτύπου. Αυτό επιτρέπει στο *Horizontal Pod Autoscaler (HPA)* που έχει ρυθμιστεί για το *backend* να αυξάνει ή να μειώνει δυναμικά τον αριθμό των αντιτύπων με βάση τον φόρτο, χωρίς κίνδυνο απώλειας πληροφορίας ή ασυνέπειας μεταξύ χρηστών.

Τα σχεδιαστικά πρότυπα που παρουσιάστηκαν — η αποσύνδεση μεγάλου περιεχομένου από τα μηνύματα, η ανθεκτικότητα σε προσωρινές αποτυχίες δικτύου, η προστασία από διπλές εγγραφές, και η στατελής φύση του *backend* — δεν λειτουργούν μεμονωμένα, αλλά συνθέτουν ένα σύστημα που παραμένει λειτουργικό υπό συνθήκες που, σε διαφορετική περίπτωση, θα οδηγούσαν σε σφάλματα ή ασυνέπειες δεδομένων. Το επόμενο κεφάλαιο εξετάζει ένα ακόμη τμήμα του συστήματος που υλοποιεί διαφορετική φιλοσοφία υπολογισμού: τη *serverless* εκτέλεση κώδικα, ενεργοποιούμενη αποκλειστικά από συγκεκριμένα γεγονότα.

6 Κεφάλαιο 6: Serverless Υπολογισμοί και λογική αυτοματοποίησης

Όχι κάθε λειτουργία του συστήματος χρειάζεται μόνιμα ενεργό υπολογιστικό πόρο. Το παρόν κεφάλαιο παρουσιάζει την υλοποίηση μιας *serverless* συνάρτησης, που ενεργοποιείται αποκλειστικά όταν συμβαίνει ένα συγκεκριμένο γεγονός, και αποδεσμεύει πλήρως τους πόρους της όταν δεν χρησιμοποιείται — καθώς και ένα πραγματικό πρόβλημα που προέκυψε κατά την υλοποίηση, η διάγνωση και επίλυση του οποίου αποδεικνύει στην πράξη τις ιδιαιτερότητες του *event-driven* προγραμματιστικού μοντέλου. Σκοπός: Event-driven λειτουργίες για οικονομία πόρων.

6.1 Εγκατάσταση *serverless* πλατφόρμας: *Knative*

Για την υλοποίηση *serverless* υπολογισμού επιλέχθηκε το *Knative Serving*, αντί εναλλακτικών όπως το *OpenWhisk* ή υπηρεσίες *managed serverless* δημόσιου *cloud* (π.χ. *AWS Lambda*). Η επιλογή αυτή έγινε επειδή το *Knative* λειτουργεί **εντός** του υπάρχοντος *Kubernetes cluster* (*MicroK8s*), χωρίς να απαιτεί ξεχωριστή υποδομή ή εξάρτηση από συγκεκριμένο πάροχο *cloud*, διατηρώντας έτσι τη συνολική αρχιτεκτονική συνεπή με τις υπόλοιπες υπηρεσίες του συστήματος, οι οποίες ήδη λειτουργούν ως πόροι *Kubernetes*. Το *OpenWhisk* θα απαιτούσε ξεχωριστό *cluster* ή *namespace* αφιερωμένο στη διαχείρισή του, ενώ το *AWS Lambda* θα εισήγαγε εξάρτηση από συγκεκριμένο πάροχο (*vendor lock-in*) και πρόσθετη πολυπλοκότητα δικτύωσης για την επικοινωνία με το *MinIO*, το οποίο παραμένει αυτο-φιλοξενούμενο εντός του *cluster*.

Η ενεργοποίηση του *Knative* πραγματοποιήθηκε ως πρόσθετο (*addon*) του *MicroK8s*, αυτοματοποιημένη μέσω αφιερωμένου *role* του *Ansible*:

```
- name: Enable MicroK8s community addons repository
  command: microk8s enable community

- name: Enable MicroK8s Knative addon
  command: microk8s enable knative
```

Listing 13: Ansible tasks για την ενεργοποίηση των *MicroK8s* addons

6.2 Σχεδιασμός *triggering* μηχανισμού

Η ενεργοποίηση της *serverless* συνάρτησης βασίζεται στον μηχανισμό ειδοποίησης γεγονότων (*event notification*) του *MinIO*: κάθε φορά που αποθηκεύεται νέο αντικείμενο σε συγκεκριμένο *bucket*, το *MinIO* αποστέλλει αυτόματα *webhook* ειδοποίηση προς προκαθορισμένο *endpoint* — στην προκειμένη περίπτωση, προς τη διεύθυνση της συνάρτησης *Knative*:

```
mc event add myminio/knative-demo \
  arn:minio:sqs::thumbnailfunction:webhook \
  --event put
```

Listing 14: Εγγραφή event notification στον *MinIO* client

Το *Knative Serving* αναλαμβάνει τη διαχείριση του κύκλου ζωής της συνάρτησης: όταν δεν υπάρχει εισερχόμενη κίνηση για ορισμένο διάστημα, το *Knative* μειώνει τον αριθμό των

αντιτύπων (*replicas*) στο μηδέν (*scale-to-zero*), αποδεδμεύοντας πλήρως τους υπολογιστικούς πόρους. Με την άφιξη νέου γεγονότος, δημιουργείται αυτόματα νέο αντίτυπο (*cold start*) για την εξυπηρέτησή του.

6.3 Εκτέλεση επιχειρηματικής λογικής: (*thumbnails*)

Η συνάρτηση που υλοποιήθηκε εκτελεί τη δημιουργία μικρογραφίας (*thumbnail*) για κάθε νέα εικόνα που αποθηκεύεται στο παρακολουθούμενο *bucket*: λαμβάνει το αρχείο, το μετασχηματίζει σε μικρότερες διαστάσεις μέσω της βιβλιοθήκης *sharp*, και αποθηκεύει το αποτέλεσμα πίσω στο *MinIO*, υπό όνομα που υποδεικνύει ότι πρόκειται για παράγωγο αρχείο (πρόθεμα *thumb-*):

```
const thumbBuffer = await sharp(inputBuffer)
  .resize(200, 200, { fit: 'inside' })
  .jpeg()
  .toBuffer();

const thumbKey = `thumb-${key}`;
await minioClient.putObject(bucket, thumbKey, thumbBuffer, ...);
```

Listing 15: Επεξεργασία εικόνας με Sharp για δημιουργία thumbnail

6.3.1 Διάγνωση και επίλυση *infinite loop* σε *event-driven* λογική

Κατά τη δοκιμή της συνάρτησης εντοπίστηκε σοβαρό πρόβλημα: η συνάρτηση εκτελούνταν ατέρμονα για το ίδιο αρχείο, χωρίς ποτέ να σταματά. Η αρχική υπόθεση ήταν ότι η αιτία βρισκόταν στο βήμα επισήμανσης (*tagging*) του αρχείου ως ήδη επεξεργασμένου, μέσω της κλήσης *setObjectTagging()*, η οποία υποτίθεται ότι θα πυροδοτούσε νέο γεγονός στο *MinIO*.

Πριν επιχειρηθεί οποιαδήποτε διόρθωση στον κώδικα, αφαιρέθηκε αμέσως ο μηχανισμός ειδοποίησης (*event notification*) από το *bucket*, ώστε να σταματήσει η κατανάλωση πόρων χωρίς να γίνει παρέμβαση υπό πίεση χρόνου. Η διάγνωση που ακολούθησε, μέσω παρακολούθησης των αρχείων καταγραφής (*logs*) σε πραγματικό χρόνο, επιβεβαίωσε την αρχική υπόθεση: η *setObjectTagging()* πράγματι παράγει ένα event όπου είναι της μορφής *s3:ObjectCreated:PutTagging*, το οποίο ενεργοποιεί εκ νέου την ίδια συνάρτηση για το ίδιο αρχείο.

Ο μηχανισμός προστασίας από επανεπεξεργασία απέτυχε σιωπηλά εξαιτίας ασυμβατότητας τύπων δεδομένων: ο κώδικας έλεγχε αν η τιμή της επισήμανσης ήταν η συμβολοσειρά (*string*) *'true'*, ενώ η βιβλιοθήκη *minio-js* επιστρέφει τη **λογική** (*boolean*) τιμή *true*:

```
// Before (fails silently):
if (tags.some(t => t.Key === 'processed' && t.Value === 'true'))

// Next (checks both possible forms):
if (tags.some(t => t.Key === 'processed' &&
  (t.Value === 'true' || t.Value === true)))
```

Listing 16: Διόρθωση ελέγχου tags για την αποφυγή silent failures

Η διόρθωση επιβεβαιώθηκε μέσω νέας δοκιμής, με ταυτόχρονη παρακολούθηση των *logs* σε πραγματικό χρόνο: το αρχείο επεξεργάστηκε επιτυχώς ακριβώς μία φορά, και το επόμενο γεγονός (*PutTagging*) αναγνωρίστηκε σωστά ως ήδη επεξεργασμένο, χωρίς νέο κύκλο επεξεργασίας. Το περιστατικό αυτό αναδεικνύει μια σημαντική ιδιαιτερότητα των *event-driven* αρχιτεκτονικών: μια ενέργεια γραφής — όπως η επισήμανση ενός αρχείου — μπορεί η ίδια να παράγει νέο γεγονός, και σφάλματα σύγκρισης τύπων σε συνθήκες προστασίας (*guard conditions*) δεν εκδηλώνονται ως κατάρρευση, αλλά ως ανόρθωση, ατέρμονη επανάληψη.

```

azureuser@fabric-vm:~/fabric-smart-warehouse/serverless/thumbnail-function$ microk8s kubectl logs -l serving.k
2026-06-20T01:05:51.854742315Z DEBUG tags shape: []
2026-06-20T01:05:51.901018323Z Received request body: {"eventName":"s3:ObjectCreated:Put","key":"knative-demo/
":"2026-06-20T01:05:51.892Z","eventName":"s3:ObjectCreated:Put","userIdentity":{"principalId":"admin"},"reques
id-2":"dd9025bab4ad464b049177c95eb6ebf374d3b3fd1af9251148b658df7ac2e3e8","x-amz-request-id":"18BAA4928AF5EE8D"
7:9000"},"s3":{"s3SchemaVersion":"1.0","configurationId":"Config","bucket":{"name":"knative-demo","ownerIdenti
ze":1478,"eTag":"55eee91e6364d05035405674ea51f243","contentType":"image/jpeg","userMetadata":{"content-type":"
inux; x64) minio-js/8.0.7"}}}}
2026-06-20T01:05:51.910776797Z ✓ Thumbnail created: thumb-fresh-hua-test.png
2026-06-20T01:05:51.911785333Z Received request body: {"eventName":"s3:ObjectCreated:PutTagging","key":"knativ
e":"2026-06-20T01:05:51.910Z","eventName":"s3:ObjectCreated:PutTagging","userIdentity":{"principalId":"admin"}
{"content-length":"0","x-amz-id-2":"dd9025bab4ad464b049177c95eb6ebf374d3b3fd1af9251148b658df7ac2e3e8","x-amz-r
endpoint":"http://10.1.253.247:9000"},"s3":{"s3SchemaVersion":"1.0","configurationId":"Config","bucket":{"name
fresh-hua-test.png","size":43039,"eTag":"98426026b60bb9ec036957420501b8da","contentType":"image/png","userMeta
,"userAgent":"MinIO (linux; x64) minio-js/8.0.7"}}}}
2026-06-20T01:05:51.915821470Z DEBUG tags shape: [{"key":"processed","value":true}, {"key":"thumbnail","value":
2026-06-20T01:05:51.915829756Z Already processed: fresh-hua-test.png
azureuser@fabric-vm:~/fabric-smart-warehouse/serverless/thumbnail-function$

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 9

● azureuser@fabric-vm:~/fabric-smart-warehouse/serverless/thumbnail-function$ mc ls myminio/products/
[2026-06-18 13:00:32 UTC] 20KiB STANDARD img-1781787632108-levis.jpeg
[2026-06-18 14:29:23 UTC] 20KiB STANDARD img-1781792963310-levis.jpeg
[2026-06-19 21:49:23 UTC] 20KiB STANDARD img-1781905763631-levis.jpeg
[2026-06-19 21:57:37 UTC] 20KiB STANDARD img-1781906257471-levis.jpeg
[2026-06-19 22:09:17 UTC] 20KiB STANDARD img-1781906957574-levis.jpeg
[2026-06-20 01:01:05 UTC] 42KiB STANDARD img-1781917265851-hua-logo.png
[2026-06-19 19:45:58 UTC] 3.1KiB STANDARD thumb-img-1781787632108-levis.jpeg
● azureuser@fabric-vm:~/fabric-smart-warehouse/serverless/thumbnail-function$ mc cp myminio/products/img-1781917
...ucts/img-1781917265851-hua-logo.png: 42.03 KiB / 42.03 KiB
● azureuser@fabric-vm:~/fabric-smart-warehouse/serverless/thumbnail-function$ sleep 10 && microk8s kubectl logs
2026-06-20T01:05:51.901018323Z Received request body: {"eventName":"s3:ObjectCreated:Put","key":"knative-demo/
":"2026-06-20T01:05:51.892Z","eventName":"s3:ObjectCreated:Put","userIdentity":{"principalId":"admin"},"reques
id-2":"dd9025bab4ad464b049177c95eb6ebf374d3b3fd1af9251148b658df7ac2e3e8","x-amz-request-id":"18BAA4928AF5EE8D"
7:9000"},"s3":{"s3SchemaVersion":"1.0","configurationId":"Config","bucket":{"name":"knative-demo","ownerIdenti
ze":1478,"eTag":"55eee91e6364d05035405674ea51f243","contentType":"image/jpeg","userMetadata":{"content-type":"
inux; x64) minio-js/8.0.7"}}}}
2026-06-20T01:05:51.910776797Z ✓ Thumbnail created: thumb-fresh-hua-test.png
2026-06-20T01:05:51.911785333Z Received request body: {"eventName":"s3:ObjectCreated:PutTagging","key":"knativ
e":"2026-06-20T01:05:51.910Z","eventName":"s3:ObjectCreated:PutTagging","userIdentity":{"principalId":"admin"}
{"content-length":"0","x-amz-id-2":"dd9025bab4ad464b049177c95eb6ebf374d3b3fd1af9251148b658df7ac2e3e8","x-amz-r
endpoint":"http://10.1.253.247:9000"},"s3":{"s3SchemaVersion":"1.0","configurationId":"Config","bucket":{"name
fresh-hua-test.png","size":43039,"eTag":"98426026b60bb9ec036957420501b8da","contentType":"image/png","userMeta
,"userAgent":"MinIO (linux; x64) minio-js/8.0.7"}}}}
2026-06-20T01:05:51.915821470Z DEBUG tags shape: [{"key":"processed","value":true}, {"key":"thumbnail","value":
2026-06-20T01:05:51.915829756Z Already processed: fresh-hua-test.png
azureuser@fabric-vm:~/fabric-smart-warehouse/serverless/thumbnail-function$

```

Εικόνα 6.1: Αρχεία καταγραφής (*logs*) της συνάρτησης *Knative* μετά τη διόρθωση: το αρχείο επεξεργάζεται μία φορά («*Thumbnail created*»), και το επόμενο γεγονός *PutTagging* αναγνωρίζεται σωστά ως ήδη επεξεργασμένο («*Already processed*»), χωρίς νέο κύκλο εκτέλεσης

Το περιστατικό αυτό αναδεικνύει μια σημαντική ιδιαιτερότητα των *event-driven* αρχιτεκτονικών: μια ενέργεια γραφής — όπως η επισήμανση ενός αρχείου — μπορεί η ίδια να παράγει νέο γεγονός, και σφάλματα σύγκρισης τύπων σε συνθήκες προστασίας (*guard conditions*) δεν εκδηλώνονται ως κατάρρευση, αλλά ως ανόρθωση, ατέρμονη επανάληψη. Ακριβώς επειδή η αποτυχία ήταν σιωπηλή, η ανίχνευσή της δεν θα ήταν δυνατή μέσα από στατική ανάλυση κώδικα — μόνο η συστηματική παρακολούθηση της πραγματικής συμπεριφοράς, σε πραγματικό χρόνο εκτέλεσης, αποκάλυψε το πρόβλημα. Το επόμενο κεφάλαιο επεκτείνει αυτή την ανάγκη παρακολούθησης σε επίπεδο ολόκληρου του συστήματος, παρουσιάζοντας τα εργαλεία παρατηρητικότητας (*observability*), τον μηχανισμό αυτόματης κλιμάκωσης, και τις πειραματικές μετρήσεις που οδήγησαν στον καθορισμό επιπέδων ποιότητας υπηρεσίας.

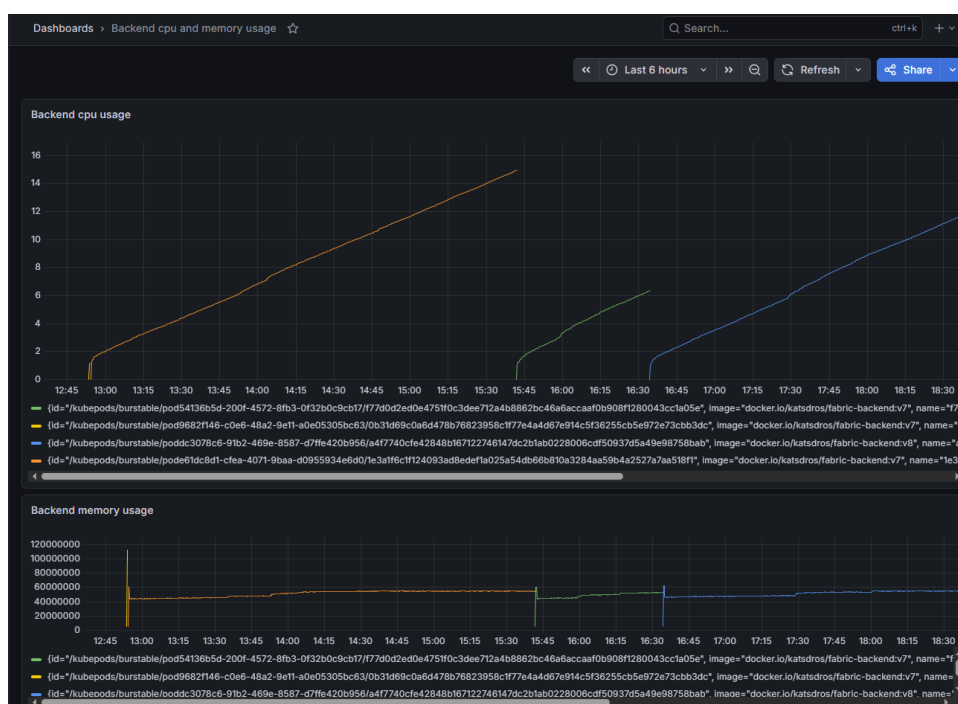
7 Κεφάλαιο 7: Παρακολούθηση, κλιμάκωση και SLAs

Η ανθεκτικότητα του κώδικα και η αυτοματοποίηση της ανάπτυξης, όπως παρουσιάστηκαν στα προηγούμενα κεφάλαια, δεν αρκούν από μόνες τους για να εγγυηθούν ότι το σύστημα ανταποκρίνεται στις απαιτήσεις φόρτου που μπορεί να αντιμετωπίσει στην πράξη. Το παρόν κεφάλαιο παρουσιάζει τον τρόπο με τον οποίο το σύστημα παρακολουθείται σε πραγματικό χρόνο, τον μηχανισμό αυτόματης κλιμάκωσης που ανταποκρίνεται σε αυξημένο φόρτο, και τα αποτελέσματα πειραματικών δοκιμών φόρτου που οδήγησαν στον καθορισμό συγκεκριμένων, μετρήσιμων επιπέδων ποιότητας υπηρεσίας (*SLA*).

7.1 Οπτικοποίηση και μετρικές (*Observability*): *Prometheus* & *Grafana*

Η παρακολούθηση της απόδοσης ενός συστήματος εν λειτουργία προϋποθέτει τη συλλογή και την οπτικοποίηση μετρικών σε πραγματικό χρόνο. Για τον σκοπό αυτό εγκαταστάθηκε ο συνδυασμός *Prometheus* (συλλογή και αποθήκευση μετρικών χρονοσειρών) και *Grafana* (οπτικοποίηση), μέσω του πακέτου *kube-prometheus-stack*, αυτοματοποιημένου μέσω *Ansible*.

Οι μετρικές που παρακολουθούνται περιλαμβάνουν τη χρήση επεξεργαστή (*CPU*) και μνήμης (*memory*) ανά *pod*, καθώς και τον αριθμό αιτημάτων *HTTP* που εξυπηρετούνται από τον *backend*. Αυτές οι δύο κατηγορίες μετρικών αποτελούν τη βάση για τις αποφάσεις αυτόματης κλιμάκωσης που περιγράφονται στην επόμενη ενότητα. Η [Figure 7.1](#) δείχνει ένα προσαρμοσμένο *dashboard* που παρακολουθεί ειδικά τη χρήση *CPU* και μνήμης του *backend pod*, καθώς και του *sidecar container* του *Vault Agent*.

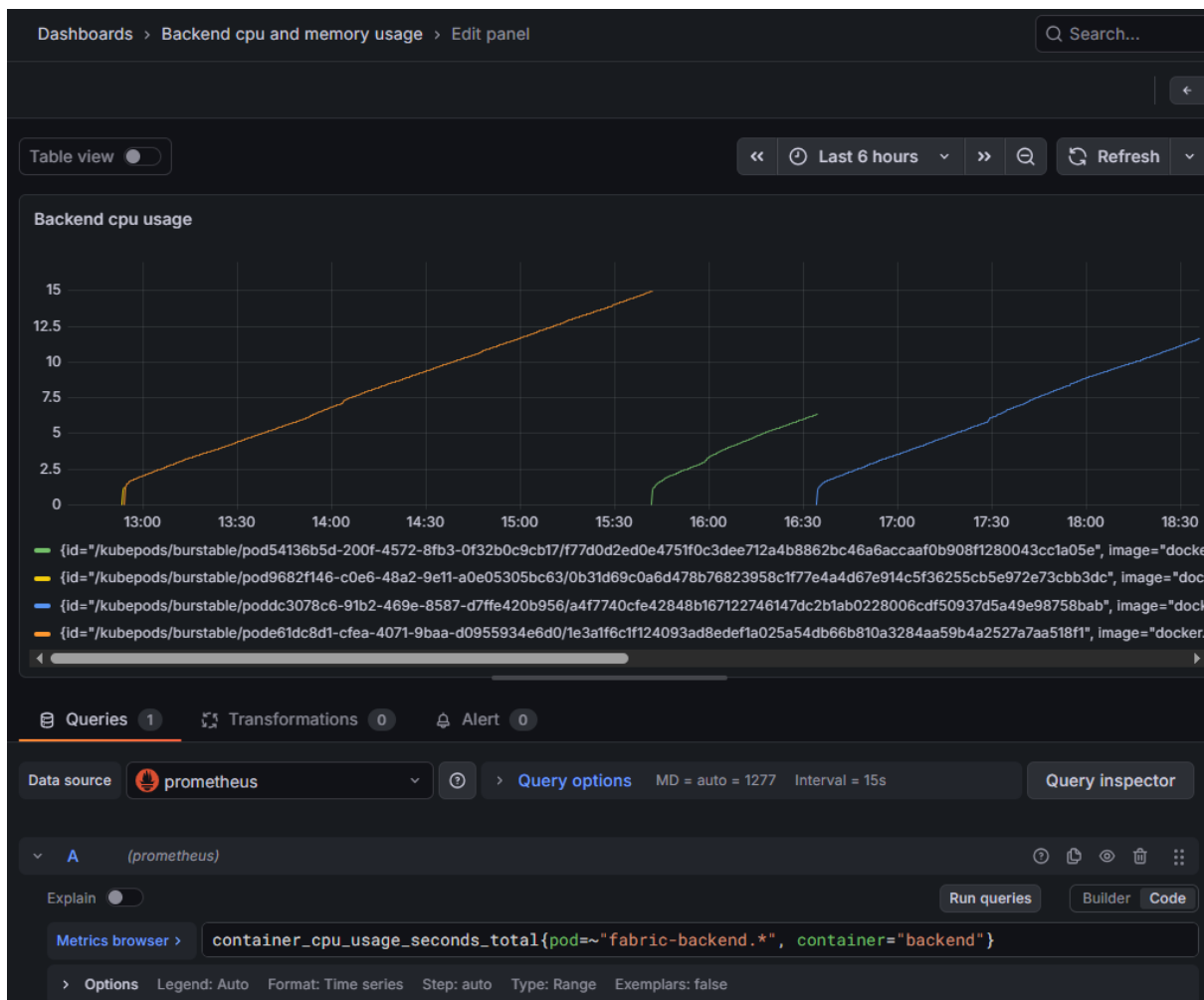


Εικόνα 7.1: *Grafana*: προσαρμοσμένο *dashboard* παρακολούθησης *CPU* και μνήμης

Η μετρική χρήσης *CPU* που τροφοδοτεί τις αποφάσεις αυτόματης κλιμάκωσης (subsection 7.2) αναχτάται μέσω *PromQL* που φιλτράρει αποκλειστικά τα *pod*s του *backend*:

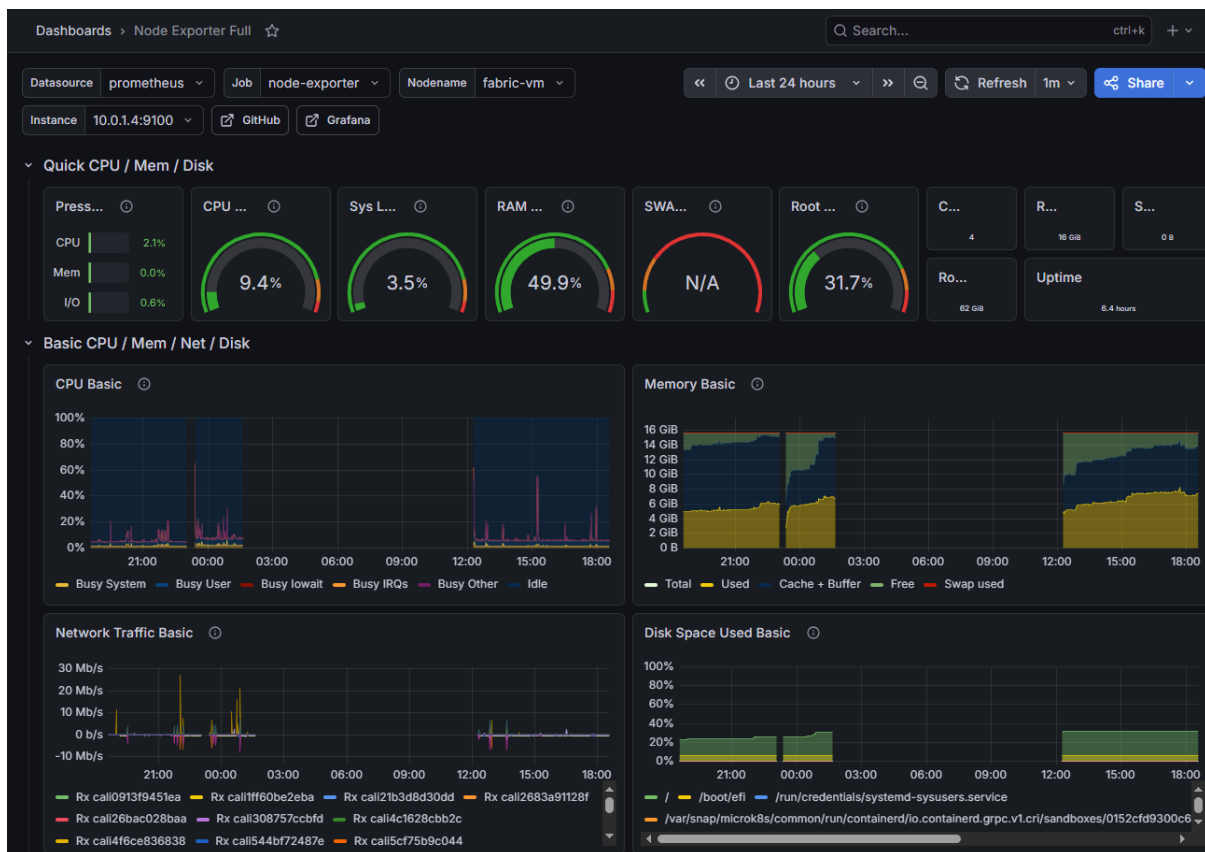
```
container_cpu_usage_seconds_total{pod=~"fabric-backend.*",  
  container="backend"}
```

Listing 17: PromQL query για τη χρήση CPU του backend



Εικόνα 7.2: Grafana: προβολή επεξεργασίας (*edit panel*) του ερωτήματος *PromQL* για τη χρήση *CPU* του *backend*

Πέρα από τις μετρικές ανά *pod*, είναι χρήσιμο να επιβεβαιωθεί και η συνολική κατάσταση της εικονικής μηχανής που φιλοξενεί ολόκληρο το *cluster*. Για τον σκοπό αυτό χρησιμοποιήθηκε το έτοιμο (*community*) dashboard "*Node Exporter Full*", το οποίο παρέχει μια συνολική εικόνα χρήσης *CPU*, μνήμης, δικτύου, και αποθηκευτικού χώρου σε επίπεδο κόμβου (*node-level*), ανεξάρτητα από τον καταμερισμό σε επιμέρους *pod*s - (Εικόνα 7.3). Όπως φαίνεται, η συνολική χρήση παραμένει σε υγιή επίπεδα (*CPU* 9.4%, μνήμη 49.9%) παρά το γεγονός ότι η εικονική μηχανή φιλοξενεί ταυτόχρονα το σύνολο των υπηρεσιών του συστήματος — *Jenkins*, *ArgoCD*, *Vault*, και την ίδια την εφαρμογή.



Εικόνα 7.3: Grafana: dashboard "Node Exporter Full" — συνολική χρήση πόρων της εικονικής μηχανής fabric-vm

7.2 Αυτόματη κλιμάκωση (HPA — Horizontal Pod Autoscaler)

Με βάση τις μετρικές χρήσης πόρων που συλλέγονται από το *Prometheus*, το *backend* είναι ρυθμισμένο με *Horizontal Pod Autoscaler*: όταν η μέση χρήση πόρων των υπάρχοντων αντιτύπων (*replicas*) υπερβαίνει προκαθορισμένο όριο, το *Kubernetes* δημιουργεί αυτόματα νέα αντίτυπα, μέχρι ένα ανώτατο όριο, κατανέμοντας τον εισερχόμενο φόρτο. Όταν ο φόρτος υποχωρεί, ο αριθμός των αντιτύπων μειώνεται ανάλογα, μέχρι ένα ελάχιστο όριο που διασφαλίζει συνεχή διαθεσιμότητα.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: fabric-backend-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: fabric-backend
  minReplicas: 1
  maxReplicas: 5
  metrics:
    - type: Resource
      resource:
```

```

    name: cpu
    target:
      type: Utilization
      averageUtilization: 50
- type: Resource
  resource:
    name: memory
    target:
      type: Utilization
      averageUtilization: 70

```

Listing 18: HorizontalPodAutoscaler manifest για το backend

Η ρύθμιση αυτή ορίζει δύο ανεξάρτητες μετρικές αναφοράς, και όχι μόνο *CPU*: το *Kubernetes* υπολογίζει τον επιθυμητό αριθμό αντιτύπων ξεχωριστά για κάθε μετρική (*CPU* στο 50% μέσης χρήσης, μνήμη στο 70%) και υιοθετεί το **μεγαλύτερο** από τα δύο αποτελέσματα — διασφαλίζοντας έτσι κλιμάκωση ακόμα και σε σενάρια όπου η πίεση προέρχεται κυρίως από τη μνήμη, χωρίς αντίστοιχη αύξηση στη χρήση *CPU* (όπως θα συνέβαινε, για παράδειγμα, σε ταυτόχρονη επεξεργασία πολλών μεγάλων αρχείων εικόνας). Ο αριθμός των αντιτύπων κυμαίνεται μεταξύ ενός ελάχιστου αντιτύπου, για συνεχή διαθεσιμότητα της υπηρεσίας, και πέντε αντιτύπων, ως ανώτατο όριο κλιμάκωσης.

Ο μηχανισμός αυτός λειτουργεί αξιόπιστα ακριβώς επειδή ο *backend* είναι σχεδιασμένος ως *stateless*, βλ. Κεφάλαιο 5): κάθε νέο αντίτυπο μπορεί να εξυπηρετήσει αμέσως αιτήματα, χωρίς ανάγκη συγχρονισμού κατάστασης με τα υπάρχοντα αντίτυπα.

7.3 Πειραματικές μετρήσεις και *load testing*

Για την παραγωγή ελεγχόμενου φόρτου χρησιμοποιήθηκε το εργαλείο *k6*, με ξεχωριστά σενάρια δοκιμής για τις δύο κύριες λειτουργίες του *API*: ανάκτηση των προϊόντων (*GET /api/products*) και δημιουργία νέου προϊόντος (*POST /api/products*). Κάθε σενάριο κλιμακώνει σταδιακά τον αριθμό ταυτόχρονων εικονικών χρηστών (*virtual users*) σε πολλαπλά στάδια, προσομοιώνοντας ρεαλιστική αυξομείωση φόρτου, με περίοδο ελεγχόμενης αποκλιμάκωσης (*graceful ramp-down*) στο τέλος κάθε δοκιμής.

Το σενάριο *GET* έφτασε έως 100 ταυτόχρονους χρήστες, με 6.823 συνολικά αιτήματα και μηδενικό ποσοστό αποτυχίας:

```

http_req_duration: avg=11.23ms p(95)=14.2ms p(99)=23.2ms
http_req_failed:    0.00% (0 out of 6823)

```

Listing 19: Αποτελέσματα μέτρησης απόδοσης (Load Test Metrics)

```

scenarios: (100.00%) 1 scenario, 100 max VUs, 3m30s max duration (incl. graceful stop):
    * default: Up to 100 looping VUs for 3m0s over 6 stages (gracefulRampDown: 30s, gracefulStop: 30s)

THRESHOLDS

http_req_duration
✓ 'p(95)<100' p(95)=14.2ms
✓ 'p(95)<250' p(95)=14.2ms
✓ 'p(99)<500' p(99)=23.2ms

http_req_failed
✓ 'rate<0.01' rate=0.00%

TOTAL RESULTS

checks_total.....: 6823    37.794773/s
checks_succeeded...: 100.00% 6823 out of 6823
checks_failed.....: 0.00%   0 out of 6823

✓ status is 200

HTTP
http_req_duration.....: avg=11.23ms min=9.2ms med=10.5ms max=138.43ms p(90)=11.96ms p(95)=14.2ms
  { expected_response:true }...: avg=11.23ms min=9.2ms med=10.5ms max=138.43ms p(90)=11.96ms p(95)=14.2ms
http_req_failed.....: 0.00%   0 out of 6823
http_reqs.....: 6823    37.794773/s

EXECUTION
iteration_duration.....: avg=1.01s min=1s med=1.01s max=1.13s p(90)=1.01s p(95)=1.01s
iterations.....: 6823    37.794773/s
vus.....: 3 min=1 max=100
vus_max.....: 100 min=100 max=100

NETWORK
data_received.....: 3.7 MB 20 kB/s
data_sent.....: 566 kB 3.1 kB/s

running (3m00.5s), 000/100 VUs, 6823 complete and 0 interrupted iterations
default ✓ [=====] 000/100 VUs 3m0s
azureuser@fabric-vm:~/fabric-smart-warehouse/ansible$

```

Εικόνα 7.4: Αποτελέσματα δοκιμής φόρτου *k6* για το *endpoint* GET /api/products: 6.823 αιτήματα, p(95)=14.2ms, μηδενικό ποσοστό αποτυχίας

Το σενάριο POST, λόγω της επιπλέον επιβάρυνσης από την εγγραφή στη βάση δεδομένων και τη μεταφόρτωση αρχείου στο *MinIO*, έφτασε έως 20 ταυτόχρονους χρήστες, με 1.018 συνολικά αιτήματα και εξίσου μηδενικό ποσοστό αποτυχίας:

```

http_req_duration: avg=70.85ms p(95)=80.13ms p(99)=156.76ms
http_req_failed: 0.00% (0 out of 1018)

```

Listing 20: Αποτελέσματα μέτρησης απόδοσης για το σύστημα

```

azureuser@fabric-vm:~/fabric-smart-warehouse$ k6 run ~/fabric-smart-warehouse/load-tests/products-post-test.js
execution: local
script: /home/azureuser/fabric-smart-warehouse/load-tests/products-post-test.js
output: -

scenarios: (100.00%) 1 scenario, 20 max VUs, 2m30s max duration (incl. graceful stop):
  * default: Up to 20 looping VUs for 2m0s over 4 stages (gracefulRampDown: 30s, gracefulStop: 30s)

THRESHOLDS

http_req_duration
✓ 'p(95)<500' p(95)=80.13ms
✓ 'p(99)<1000' p(99)=156.76ms

http_req_failed
✓ 'rate<0.05' rate=0.00%

TOTAL RESULTS

checks_total.....: 1018      8.464591/s
checks_succeeded...: 100.00% 1018 out of 1018
checks_failed.....: 0.00%    0 out of 1018

✓ status is 201
✓ response time < 1000ms

HTTP
http_req_duration.....: avg=70.85ms min=63.67ms med=67.74ms max=307.14ms p(90)=75.52ms p(95)=80.13ms
  { expected_response:true }...: avg=70.85ms min=63.67ms med=67.74ms max=307.14ms p(90)=75.52ms p(95)=80.13ms
http_req_failed.....: 0.00%    0 out of 509
http_reqs.....: 509      4.232296/s

EXECUTION
iteration_duration.....: avg=2.07s   min=2.06s   med=2.06s   max=2.3s    p(90)=2.07s p(95)=2.08s
iterations.....: 509      4.232296/s
vus.....: 1          min=1      max=20
vus_max.....: 20       min=20     max=20

NETWORK
data_received.....: 298 kB 2.5 kB/s
data_sent.....: 579 kB 4.8 kB/s

running (2m00.3s), 00/20 VUs, 509 complete and 0 interrupted iterations
default ✓ [=====] 00/20 VUs  2m0s
azureuser@fabric-vm:~/fabric-smart-warehouse$

```

Εικόνα 7.5: Αποτελέσματα δοκιμής φόρτου *k6* για το *endpoint* POST */api/products*: 1.018 αιτήματα, p(95)=80.13ms, μηδενικό ποσοστό αποτυχίας

Αξίζει να σημειωθεί ένας περιορισμός που εντοπίστηκε κατά τη διάρκεια προγενέστερων δοκιμών: σε σενάριο όπου η βάση δεδομένων είχε ήδη συγκεντρώσει μεγάλο αριθμό δοκιμαστικών εγγράφων (από επαναλαμβανόμενες εκτελέσεις του ίδιου *load test*), ο χρόνος απόκρισης του GET */api/products* υποβαθμίστηκε σημαντικά υπό υψηλό φόρτο (p(95) στα όρια των 5 δευτερολέπτων στα 100 *virtual users*), καθώς το *endpoint* επιστρέφει το σύνολο των εγγράφων χωρίς *pagination*. Σε περιβάλλον παραγωγής, η συμπεριφορά αυτή θα αντιμετωπιζόταν με την εφαρμογή *pagination* ή *caching* στο επίπεδο του *API*, ώστε ο χρόνος απόκρισης να παραμένει ανεξάρτητος από το συνολικό μέγεθος της συλλογής δεδομένων.

```

azureuser@fabric-vm:~/fabric-smart-warehouse$ k6 run ~/fabric-smart-warehouse/load-tests/products-post-test.js
  execution: local
    script: /home/azureuser/fabric-smart-warehouse/load-tests/products-post-test.js
    output: -

  scenarios: (100.00%) 1 scenario, 20 max VUs, 2m30s max duration (incl. graceful stop):
    * default: Up to 20 looping VUs for 2m0s over 4 stages (gracefulRampDown: 30s, gracefulStop: 30s)

  THRESHOLDS

    http_req_duration
    ✓ 'p(95)<500' p(95)=1.5ms
    ✓ 'p(99)<1000' p(99)=1.91ms

    http_req_failed
    ✗ 'rate<0.05' rate=100.00%

  TOTAL RESULTS

    checks_total.....: 1048    8.724645/s
    checks_succeeded...: 50.00% 524 out of 1048
    checks_failed.....: 50.00% 524 out of 1048

    ✗ status is 201
    ↳ 0% - ✓ 0 / ✗ 524
    ✓ response time < 1000ms

  HTTP
    http_req_duration....: avg=1.04ms min=574.96µs med=1ms max=5.48ms p(90)=1.4ms p(95)=1.5ms
    http_req_failed.....: 100.00% 524 out of 524
    http_reqs.....: 524    4.362323/s

  EXECUTION
    iteration_duration...: avg=2s    min=2s    med=2s    max=2s    p(90)=2s    p(95)=2s
    iterations.....: 524    4.362323/s
    vus.....: 1    min=1    max=20
    vus_max.....: 20    min=20    max=20

  NETWORK
    data_received.....: 163 kB    1.4 kB/s
    data_sent.....: 121 kB    1.0 kB/s

  running (2m00.1s), 00/20 VUs, 524 complete and 0 interrupted iterations
  default ✓ [=====] 00/20 VUs 2m0s
  ERRO[0120] thresholds on metrics 'http_req_failed' have been crossed

```

Εικόνα 7.6: Αποτελέσματα δοκιμής φόρτου *k6* για το *endpoint* POST */api/products*: 1.018 αιτήματα, p(95)=80.13ms, μηδενικό ποσοστό αποτυχίας

7.4 Καθορισμός επιπέδων ποιότητας υπηρεσίας (*SLA Levels*)

Οι μετρήσεις απόδοσης που παρουσιάστηκαν στην προηγούμενη ενότητα αποκτούν πρακτική αξία μόνο όταν μεταφραστούν σε συγκεκριμένες, μετρήσιμες δεσμεύσεις ποιότητας υπηρεσίας: όχι απλά «το σύστημα είναι γρήγορο», αλλά «το 95% των αιτημάτων ανάγνωσης απαντιέται σε λιγότερο από *X* χρόνο». Η ενότητα αυτή περιγράφει τη μεθοδολογία ανάλυσης που οδήγησε σε αυτές τις δεσμεύσεις, και τον ορισμό τριών διαβαθμισμένων επιπέδων (*Gold*, *Silver*, *Bronze*) βάσει των πραγματικών αποτελεσμάτων των δοκιμών φόρτου.

7.4.1 Ανάλυση κατανομής (99% CDF)

Ο καθορισμός επιπέδων ποιότητας υπηρεσίας δεν βασίστηκε σε αυθαίρετες τιμές, αλλά στην πραγματική κατανομή των χρόνων απόκρισης που καταγράφηκαν κατά τις δοκιμές φόρτου. Η μετρική $p(95)$ (95ο εκατοστημόριο) υποδεικνύει τον χρόνο απόκρισης κάτω από τον οποίο βρίσκεται το 95% των αιτημάτων, ενώ η μετρική $p(99)$ αντιστοιχεί στο αυστηρότερο όριο του 99%. Η χρήση εκατοστημορίων, αντί του μέσου όρου (*average*), αποτυπώνει πιο ρεαλιστικά την εμπειρία του χρήστη, καθώς δεν επηρεάζεται από μεμονωμένες ακραίες τιμές (*outliers*) προς τη μία ή την άλλη κατεύθυνση.

7.4.2 Ορισμός κλάσεων SLA (*Gold, Silver, Bronze*) βάσει χρόνου απόκρισης

Με βάση τις μετρημένες τιμές $p(95)$ και $p(99)$ των δύο σεναρίων δοκιμής, ορίστηκαν τρεις κλάσεις ποιότητας υπηρεσίας, εφαρμόσιμες ανά τύπο λειτουργίας (*GET* ή *POST*):

- **Gold:** χρόνος απόκρισης $p(95) < 100\text{ms}$ — κατάλληλο επίπεδο για λειτουργίες ανάγνωσης (*GET*), όπου η μετρημένη τιμή (14.2ms) βρίσκεται με μεγάλο περιθώριο εντός του ορίου.
- **Silver:** χρόνος απόκρισης $p(95) < 250\text{ms}$ — κατάλληλο επίπεδο για λειτουργίες εγγραφής (*POST*) που περιλαμβάνουν μεταφόρτωση αρχείου, όπου η μετρημένη τιμή (80.13ms) βρίσκεται εντός του ορίου, αντικατοπτρίζοντας τη φυσικά υψηλότερη επιβάρυνση αυτής της λειτουργίας.
- **Bronze:** χρόνος απόκρισης $p(99) < 500\text{ms}$ — το ευρύτερο, εγγυημένο όριο για το σύνολο σχεδόν όλων (99%) των αιτημάτων, ανεξαρτήτως τύπου, καλύπτοντας περιπτώσεις στιγμιαίας αυξημένης επιβάρυνσης.

Gold SLA $p(95) < 100\text{ms}$ (measured: 14.2ms)	Silver SLA $p(95) < 250\text{ms}$ (measured: 80.13ms)	Bronze SLA $p(99) < 500\text{ms}$ (measured: 23.2–156.76ms)
---	--	--

Συγκρίνοντας τα δύο σενάρια δοκιμής, παρατηρείται ότι η λειτουργία ανάγνωσης (*GET*) διατηρεί χρόνο απόκρισης περίπου **5,6 φορές** μικρότερο από τη λειτουργία εγγραφής (*POST*) στο $p(95)$ (14.2ms έναντι 80.13ms), διαφορά που αντικατοπτρίζει εύλογα τον πρόσθετο φόρτο εργασίας του *POST* — εγγραφή στη *MongoDB* και μεταφόρτωση αρχείου στο *MinIO* — έναντι μιας απλής ανάκτησης δεδομένων. Και οι δύο λειτουργίες, ωστόσο, παραμένουν με μεγάλο περιθώριο εντός των αντίστοιχων ορίων *SLA* που ορίστηκαν, επιβεβαιώνοντας ότι το σύστημα ανταποκρίνεται αξιόπιστα στον αναμενόμενο φόρτο, υπό τις συνθήκες δοκιμής που εξετάστηκαν. Τα τρία επίπεδα ποιότητας υπηρεσίας που ορίστηκαν σε αυτό το κεφάλαιο — σε συνδυασμό με την παρακολούθηση σε πραγματικό χρόνο και την αυτόματη κλιμάκωση που παρουσιάστηκαν προηγουμένως — συνθέτουν ένα πλήρες πλαίσιο διαχείρισης απόδοσης, ικανό όχι μόνο να μετρά τη συμπεριφορά του συστήματος, αλλά και να την εγγυάται με συγκεκριμένους, ποσοτικούς όρους.

Συμπεράσματα και μελλοντικές επεκτάσεις

Η παρούσα εργασία παρουσίασε τη μετάβαση μιας ήδη *containerized* εφαρμογής διαχείρισης αποθήκης από τοπική εγκατάσταση σε πλήρως αυτοματοποιημένη, *cloud-native* αρχιτεκτονική στο *Microsoft Azure*. Η υλοποίηση κάλυψε το σύνολο του κύκλου ζωής μιας σύγχρονης *cloud* εφαρμογής: από τη δημιουργία της υποδομής μέσω *Infrastructure as Code* (*OpenTofu*) και την αυτοματοποιημένη ρύθμισή της (*Ansible*), έως την ασφαλή διαχείριση μυστικών (*HashiCorp Vault*), τις πρακτικές *GitOps* (*Jenkins*, *ArgoCD*), την ενσωμάτωση σχεδιαστικών προτύπων ανθεκτικότητας στον κώδικα, τη *serverless* εκτέλεση συγκεκριμένων λειτουργιών (*Knative*), και την τεκμηρίωση επιπέδων ποιότητας υπηρεσίας μέσω πειραματικών δοκιμών φόρτου.

Η εμπειρία της υλοποίησης ανέδειξε ότι η πολυπλοκότητα μιας *cloud-native* αρχιτεκτονικής δεν έγκειται μόνο στην αρχική εγκατάσταση κάθε επιμέρους τεχνολογίας, αλλά κυρίως στις αλληλεπιδράσεις μεταξύ τους — όπως αναδείχθηκε χαρακτηριστικά από το περιστατικό *infinite loop* στη *serverless* συνάρτηση (Κεφάλαιο 6), όπου μια φαινομενικά απλή ενέργεια (επισήμανση αρχείου) είχε απρόβλεπτη επίδραση σε ολόκληρο τον μηχανισμό ενεργοποίησης γεγονότων. Παρά την ευρεία κάλυψη των τεχνολογιών και πρακτικών που ζητούνταν, η εργασία έχει συγκεκριμένους περιορισμούς, οι οποίοι αποτελούν φυσικές κατευθύνσεις μελλοντικής επέκτασης:

- **Διαχείριση secrets του *RabbitMQ*:** τα διαπιστευτήρια του *RabbitMQ* καταγράφονται στο *Vault* για λόγους συνέπειας, αλλά η πραγματική σύνδεση παραμένει βασισμένη σε μεταβλητές περιβάλλοντος, λόγω τεκμηριωμένου περιορισμού του επίσημου *Docker image* στη διαχείριση διαπιστευτηρίων μέσω μεταβλητών κατά την εκκίνηση σε πρόσφατες εκδόσεις.
- **Αυτόματη ενημέρωση εικόνας (*ArgoCD Image Updater*):** εξετάστηκε η χρήση αυτόματου μηχανισμού ενημέρωσης *image tags* χωρίς ανθρώπινη παρέμβαση, αλλά η προσέγγιση αυτή απαιτεί πηγή τύπου *Kustomize* ή *Helm* και εισάγει επιπλέον αρχεία υπερκάλυψης (*override files*) που περιπλέκουν την αναγνωσιμότητα του *repository* χωρίς αντίστοιχο όφελος στην παρούσα κλίμακα· αντί αυτού, υλοποιήθηκε αυτόματη ενημέρωση του *manifest* απευθείας από το *Jenkins pipeline*.
- **Πρότυπο *Circuit Breaker*:** δεν υλοποιήθηκε στην παρούσα φάση· θα αποτελούσε φυσική επέκταση του υπάρχοντος μηχανισμού *Retry*, προστατεύοντας το σύστημα από αλυσιδωτές αστοχίες σε περίπτωση παρατεταμένης μη διαθεσιμότητας εξωτερικής υπηρεσίας.
- **Παρακολούθηση κόστους ανά πόρο (*FinOps*):** η εκτίμηση κόστους μέσω *Infracost* καλύπτει το επίπεδο της υποδομής (*OpenTofu*), αλλά δεν επεκτείνεται σε ανάλυση κόστους ανά μεμονωμένο *pod* σε πραγματικό χρόνο — λειτουργία που θα απαιτούσε πρόσθετο εργαλείο (π.χ. *OpenCost*), εκτός του πλαισίου της παρούσας εργασίας.

Συνολικά, η εργασία απέδειξε στην πράξη ότι η μετάβαση σε *cloud-native* αρχιτεκτονική δεν είναι απλώς μεταφορά κώδικα σε διαφορετική υποδομή, αλλά υιοθέτηση ενός ολοκληρωμένου συνόλου πρακτικών — αυτοματοποίησης, ασφάλειας, παρατηρητικότητας, και σχεδιαστικής ανθεκτικότητας — που καθιστούν ένα σύστημα έτοιμο να λειτουργήσει αξιόπιστα σε πραγματικές συνθήκες παραγωγής.

Παράρτημα Α: Τεχνική αναφορά

Α.1 Πίνακας εντολών λειτουργίας (*Operational Runbook*)

Πίνακας 2: Συνοπτικός πίνακας εντολών λειτουργίας συστήματος

Action	Command
start VM	<code>az vm start -resource-group fabric-rg -name fabric-vm</code>
shutdown VM	<code>az vm deallocate -resource-group fabric-rg -name fabric-vm</code>
run Ansible playbook	<code>cd ansible && ansible-playbook -i inventory.ini playbook.yml</code>
manual unseal Vault	<code>microk8s kubectl exec -n default vault-0 - vault operator unseal <key></code>
execution load test (GET)	<code>k6 run load-tests/products-get-test.js</code>
execution load test (POST)	<code>k6 run load-tests/products-post-test.js</code>
Infrastructure cost Estimate	<code>infracost breakdown -path infrastructure/</code>

Α.2 Σημεία πρόσβασης & πηγή διαπιστευτηρίων

Πίνακας 3: Σημεία πρόσβασης στις διαχειριστικές διεπαφές του συστήματος

Service	Access Point	Credentials Source
Jenkins UI	<code>http://<VM_IP>:30808</code>	Jenkins Credentials Store
ArgoCD UI	<code>http://<VM_IP>:30880</code>	secret/fabric (Vault)
MinIO Console	<code>http://<VM_IP>:9001</code>	secret/fabric (Vault)
Grafana	<code>http://<VM_IP>:<port></code>	Helm-generated secret

A.3 Τεχνολογίες & εκδόσεις

Πίνακας 4: Κύριες τεχνολογίες υποδομής και αντίστοιχες εκδόσεις

Technology	Version
<i>MicroK8s</i>	v1.35.5 revision 8978
<i>ArgoCD</i>	v3.4.4
<i>HashiCorp Vault</i>	2.0.2
<i>Node.js</i>	20
<i>OpenTofu</i>	>= 1.6
<i>Infracost</i>	v0.10.44

A.4 Κατάλογος *Ansible roles*

Πίνακας 5: Σύνολο *Ansible roles* και η αντίστοιχη ευθύνη τους

Role	Responsibility
base-setup	<i>Docker, MicroK8s, essential addons</i>
vault	<i>HashiCorp Vault</i> installation & configuration
monitoring	<i>Prometheus & Grafana</i> setup
jenkins	<i>Jenkins</i> installation
argocd	<i>ArgoCD</i> installation
knative	<i>Knative</i> enablement, <i>MinIO Client</i> installation
infracost	Cost estimation tool installation

Παράρτημα Β: *GitHub* Repository

Για την υλοποίηση και τη διευκόλυνση της αναπαραγωγής των αποτελεσμάτων, το σύνολο των αρχείων ρυθμίσεων (*configuration files*), των *scripts* αυτοματοποίησης και του κώδικα της εφαρμογής, έχει αναρτηθεί στο προσωπικό μου *repository* στο *GitHub*.

Στο *repository* περιλαμβάνονται:

- Ο πηγαίος κώδικας του *backend* και του *frontend* (**backend/**, **frontend/**).
- Τα αρχεία υποδομής *OpenTofu* (**infrastructure/**) και τα *Ansible roles* (**ansible/**).
- Τα *Kubernetes manifests* (**kubernetes/**), το *Jenkinsfile* (**ci/**), και ο κώδικας της *serverless* συνάρτησης (**serverless/**).
- Οδηγίες εγκατάστασης και στοιχεία δοκιμαστικών λογαριασμών (**README.md**).

Πίνακας 6: Σύνδεσμοι *repository*

<i>Repository</i>	URL
Project	https://github.com/drososkats/fabric-smart-warehouse
<i>Documentation</i>	https://github.com/drososkats/fabric-smart-warehouse-docs

Βιβλιογραφικές Αναφορές

- [1] Harokopio University of Athens. *Course Materials: Distributed Systems (DIT292)*. <https://eclass.hua.gr/modules/document/?course=DIT305>. 2024.
- [2] OpenTofu. *OpenTofu Documentation*. <https://opentofu.org/docs/>. 2026.
- [3] Red Hat / Ansible. *Ansible Documentation*. <https://docs.ansible.com/>. 2026.
- [4] HashiCorp. *Vault Documentation*. <https://developer.hashicorp.com/vault/docs>. 2026.
- [5] HashiCorp. *Vault Architecture*. <https://developer.hashicorp.com/vault/docs/internals/architecture>. 2026.
- [6] Jenkins Project. *Jenkins User Documentation*. <https://www.jenkins.io/doc/>. 2026.
- [7] Argo Project. *Argo CD - Declarative GitOps CD for Kubernetes*. <https://argo-cd.readthedocs.io/>. 2026.
- [8] Knative Authors. *Knative Documentation*. <https://knative.dev/docs/>. 2026.
- [9] Canonical. *MicroK8s Documentation*. <https://microk8s.io/docs>. 2026.
- [10] Grafana Labs. *k6 Documentation*. <https://grafana.com/docs/k6/latest/>. 2026.
- [11] MinIO. *MinIO Object Storage Documentation*. <https://min.io/docs/minio/linux/index.html>. 2026.
- [12] Broadcom / RabbitMQ. *RabbitMQ Documentation*. <https://www.rabbitmq.com/docs>. 2026.
- [13] Kubernetes Authors. *Horizontal Pod Autoscaling*. <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>. 2026.
- [14] Infracost. *Infracost Documentation*. <https://www.infracost.io/docs/>. 2026.